BHy2CLI
User Guide

BHy2CLI User Guide

Document revision	3.0
Document release date	11 August 2026
Notes	Data and descriptions in this document are subject to change without notice. Product photos and pictures are for illustration purposes only and may differ from the real product appearance.

Document number: BST-MSS-SD005-30
Revision_3.0_082026

Table of Contents

List of Tables

List of Figures

Abbreviations

BHy	***BHy Smart Sensor Hub [BHIxyz]***
CLI	***Command Line Interface***
GPIO	***General Purpose Input Output***
ODR	***Output Data Rate***
PC	***Personal Computer***
RAM	***Random Access Memory***
SoC	***System on Chip***
WRD	***Wearable Reference Design***
Chip ID	***Chip Identification***
UI	***User Interface***
API	***Application Programming Interface***

Bosch Sensortec terms

Virtual sensor	Virtual sensors independently execute sensor data processing algorithms and provide software-programmed features, such as device orientation in multiple formats, gesture recognition, smart dynamic offset calibration, activity recognition, and step counting. Bosch Sensortec offers firmware versions featuring different virtual sensors to support a variety of use cases. There are 3 types of virtual sensors: continuous, on-change, one-shot.
Meta Event	The event used by the BHI360/BHI385 to notify the host about the occurrence of situations that might be of interest to the host but are not regular sensor data and they can be individually enabled or disabled
BSX	Complete 9-axis fusion solution which combines the measurements from 3-axis gyroscope, 3-axis geomagnetic sensor and a 3-axis accelerometer to provide a robust absolute orientation vector. The sensor fusion software BSX provides orientation information in form of quaternion or Euler angles.
FIFO	All virtual sensor data and associated timing information are sent to FIFO
Firmware	A linked binary image stored in a proprietary file format, digitally signed and secured through integrity verification

1 Overview

The BHy2CLI command-line interface tool is an application based on the Bosch BHy SensorAPI and COINES SDK (**Co**mmunication with **In**ertial and **E**nvironmental **S**ensors) application board framework. It is implemented by C programming language and can be used to quickly test and evaluate the BHy shuttle board with the Bosch Sensortec application board.

The core functionalities of the BHy2CLI include:

- Programming the Bhy customer shuttles
- Accessing registers
- Accessing System/Driver Specific Parameters
- Data Acquisition
- Sensor Configuration
- Accessing System/Application Status Information
- Application Configuration
- Data Injection
- Diagnostics

Figure 1: Overview

This user guide introduces the BHy2CLI commands with examples to demonstrate the use of these commands to operate the BHy Smart Sensor.

1.1 Compatibility

Table 1 describes the compatibility of BHy2CLI with other dependencies.

Item	BHy2CLI	Firmware	BHI SensorAPI	COINES SDK	Supported Boards	Supported Sensors
Version	1.3.0	3.2.0.1	2.3.1	COINES_SDK_v2.12.3	APP30 APP31	BHI360
		1.0.1.0	2.1.0			BHI385

Table 1: BHy2CLI Compatibility Matrix

2 Setup

Before using the BHy2CLI, it is necessary to understand the system design and set up the embedded environment accordingly.

2.1 Software requirements

2.1.1 COINES

As mentioned earlier, BHy2CLI is based on the COINES SDK Framework. As such, the COINES SDK environment must be set up prior to using the BHy2CLI application, since COINES installation also installs USB drivers which is necessary for MCU testing of BHY2CLI.

1. Install COINES SDK at COINES SDK | Bosch Sensortec (bosch-sensortec.com)
2. For further instructions, refer to Chapter 4, “Installation” in the COINES_SDK_User_Manual.
3. Install USB drivers to execute BHy2CLI application

2.1.2 Firmware on the Application board

The Application board ships with default firmware which may need an update depending on the time of update. In any case, it is recommended to update the board’s firmware that is available in COINES SDK.

2.1.3 BHy2CLI tool

BHy2CLI is available in Github, user can download or clone it directly.

2.1.4 BHy Compiler and Toolchain

The system compiler is for executing examples from the PC interface in COINES. The toolchain allows cross-compilation to execute examples on the Application Board’s MCU. The toolchain installation can be skipped if you only intend to run using the PC interface, for example if you just want to use the CLI tool.

2.2 Hardware requirements

This section describes the hardware required for using COINES as an evaluation tool. BHy2CLI tool requires two component which are usually delivered pre-mounted:

- Bosch Sensortec Application Board
- BHI360/BHI385 shuttle board

Figure 2: HW Setup

The BHy2CLI can be used with an Application Board to evaluate the Bhy Sensor Hubs (BHIxyz).

The Application Board refers to the Bosch Sensortec Application Board 3.0 | Bosch Sensortec (bosch-sensortec.com) and Application Board 3.1 | Bosch Sensortec (bosch-sensortec.com).

Additionally, a micro-USB-to-USB cable is required to connection the Application Board to a PC.

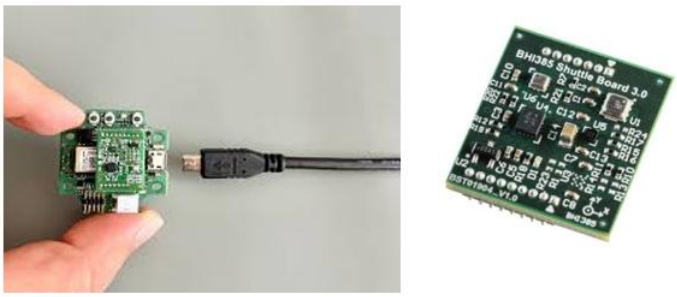

Figure 3: Required components

Table 2 shows the BHy2CLI application can be run in two operation modes.

Mode	Description
MCU	The application runs on the Application Board and interacts directly with the Sensor Hub.
PC	The application runs on a PC and interacts with the Sensor Hub via a base application running on the Application Board.

Table 2: Modes of Operation

Note: *The following instructions for the board setup are referenced for the application board.*

2.3 Building and running BHy2CLI

This section will guide you through the process of getting BHy2CLI from Github and compiling it.

2.3.1 BHy2CLI structure

Below is the release package of BHy2CLI, it holds

- **bin** : BHy2CLI and udf parser executable files to convert binary file to csv (both Windows and Linux)
- **docs**: BHy2CLI User Guide, Change log and Compatibility
- **scripts**: build/clean/download scripts
- **source:** header files and source code
- **submodules:** Sensor API packages, COINES SDK
- **LICENSE:** BHy2CLI license
- **Makefile**: a text file that defines set of tasks to be executed
- **setup_bhy2cli.sh**: A shell script to setup necessary things in Linux environment
- **README.md**: a text file that provides necessary information for user

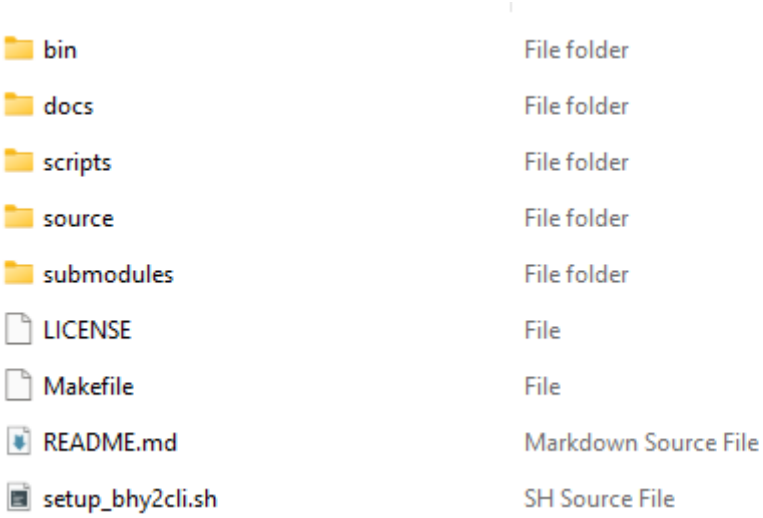

Figure 4: BHy2CLI release package

2.3.2 Cloning BHy2CLI

- Ensure GCC compiler is installed in previous part (**2.1**)
- Open local terminal (apply for both Window and Linux platform), clone BHy2CLI from Github:

git clone --recurse-submodules https://github.com/boschsensortec/BHy2CLI.git

```
$ git clone --recurse-submodules https://github.com/boschsensortec/BHy2CLI.git
Cloning into 'BHy2CLI'...
```

Figure 5: Cloning BHy2CLI

2.3.3 Compiling BHy2CLI

2.3.3.1 Window Environment

- Move to BHy2CLI folder, open Command Prompt from Windows
- For PC mode: execute **build.bat** script

```
.\scripts\build.bat
```

Figure 6: Compiling BHy2CLI for PC mode

- For MCU mode: execute **build_app30.bat** script if TARGET is MCU_APP30, **build_app31.bat** script if TARGET is MCU_APP31, **build_hear3x.bat** if TARGET is MCU_HEAR3X

```
.\scripts\build_app30.bat
```

Figure 7: Compiling BHy2CLI for MCU mode

Note: To download bin file into application board, using **download_app30.bat** if TARGET is MCU_APP30, **download_app31.bat** if TARGET is MCU_APP31, **download_hear3x.bat** if TARGET is MCU_HEAR3X

- Compilation is running

```
Platform: Windows
cc:  "C:\TDM-GCC-64\bin\gcc.exe".
Cleaning...
[ CC ] source/verbose.c
[ CC ] source/cli.c
[ CC ] source/post_mortem.c
[ CC ] source/dinject.c
[ CC ] source/common/common.c
[ CC ] source/callbacks/bhy2cli_callbacks.c
[ CC ] source/callbacks/common_callbacks.c
[ CC ] source/bhy_def/bhy_defs.c
[ CC ] submodules/bhi360/source/bhi360_event_data.c
[ CC ] submodules/bhi360/source/bhi360_bsx_algo_param.c
[ CC ] submodules/bhi360/source/bhi360.c
[ CC ] submodules/bhi360/source/bhi360_virtual_sensor_conf_param.c
[ CC ] submodules/bhi360/source/bhi360_virtual_sensor_info_param.c
[ CC ] submodules/bhi360/source/bhi360_logbin.c
[ CC ] submodules/bhi360/source/bhi360_api_entry.c
[ CC ] submodules/bhi360/source/bhi360_parse.c
[ CC ] submodules/bhi360/source/bhi360_head_orientation_param.c
[ CC ] submodules/bhi360/source/bhi360_multi_tap_param.c
[ CC ] submodules/bhi360/source/bhi360_bsec_param.c
[ CC ] submodules/bhi360/source/bhi360_hif.c
[ CC ] submodules/bhi360/source/bhi360_phy_sensor_ctrl_param.c
```

Figure 8: Compilation is running

- Verify the result to ensure it works

```
help
Usage:
bhy2cli [<port>] [<port_name>] [<options>]
<port>: optional input parameter, trigger keyword
<port_name>: optional input parameter, use it when want to support running multiple applications in parallel
Options:
  -h OR help
        = Print this usage message
  version
        = Prints the version
  -v OR verb <verbose level>
        = Set the verbose level. 0 Error, 1 Warning, 2 Infos
  -b OR ramb <firmware path>
        = Reset, upload specified firmware to RAM and boot from RAM
          [equivalent to using "reset ram <firmware> boot r" successively]
  -n OR reset
        = Reset sensor hub
  -a OR addse <sensor id>:<sensor name>:<total output payload in bytes>:
     <output_format_0>:<output_format_1>
        = Register the expected payload of a new custom virtual sensor
         -Valid output_formats:
                u8:  Unsigned 8 Bit
                u16: Unsigned 16 Bit
                u24: Unsigned 24 Bit
                u32: Unsigned 32 Bit
                s8:  Signed 8 Bit
                s16: Signed 16 Bit
                s24: Signed 24 Bit
                s32: Signed 32 Bit
                f:   Float
                c:   Char
                flb: Fixed length binary. Cannot be concatenated with other formats
                fls: Fixed length string. Cannot be concatenated with other formats
         -e.g.: addse 160:"Lean Orientation":2:fls
         -Note that the corresponding virtual sensor has to be enabled in the same function
          call (trailing actse option), since the registration of the sensor is temporary.
```

Figure 9: Verify BHy2CLI with help command

- To delete object files and executable files, execute **clean.bat** script (for MCU mode, execute **clean_app30.bat** or **clean_app31.bat**)

```
.\scripts\clean.bat
```

Figure 10: Cleaning BHy2CLI for PC mode

NOTE: In the case only download zip file of BHy2CLI, please download COINES SDK, BHI360 Sensor API, BHI385 Sensor API, then copy them into **submodules** folder of BHy2CLI.

Below is explanation for script folder in BHy2CLI package (Windows platform):

- **build.bat**: a build script for PC mode
- **build_app30.bat**: a build script for MCU mode with TARGET is Application Board 3.0
- **build_app31.bat**: a build script for MCU mode with TARGET is Application Board 3.1
- **clean_bat**: a script to delete object files and executable files for PC mode
- **clean_app30.bat**: a script to delete object files and executable files for MCU mode with TARGET is Application Board 3.0
- **clean_app31.bat**: a script to delete object files and executable files for MCU mode with TARGET is Application Board 3.1
- **download_app30.bat**: a script to download binary file for MCU mode with TARGET is Application Board 3.0
- **download_app31.bat**: a script to download binary file for MCU mode with TARGET is Application Board 3.1

2.3.3.2 Linux Environment

After downloading BHy2CLI, execute ***setup_bhy2cli.sh*** script to install necessary dependencies for compiling BHy2CLI in Linux Environment.

```
Detected distro type: DEBIAN

>>> STEP: 1. Installing build dependencies
  Updating package list...
Hit:1 https://packages-apac.osd.bosch.com/firefox noble InRelease
Hit:2 https://packages-apac.osd.bosch.com/microsoft-ubuntu-noble-prod noble InRelease
Hit:3 https://packages-apac.osd.bosch.com/edge stable InRelease
Hit:4 https://packages-apac.osd.bosch.com/vscode stable InRelease
Ign:5 https://packages-apac.osd.bosch.com/osd noble InRelease
Hit:6 https://packages-apac.osd.bosch.com/ubuntu noble InRelease
Hit:7 https://packages.microsoft.com/repos/edge-stable stable InRelease
Hit:8 https://packages-apac.osd.bosch.com/ubuntu noble-security InRelease
Hit:9 https://packages-apac.osd.bosch.com/ubuntu noble-updates InRelease
Hit:10 https://packages-apac.osd.bosch.com/osd noble Release
Reading package lists... Done
W: https://packages-apac.osd.bosch.com/firefox/dists/noble/InRelease: Signature by key 0AB215679C571D1C8325275B9BDB3D89CE49EC21 uses weak algorithm (rsa1024)
[OK] build-essential already installed
[OK] python3 already installed
[OK] libusb-1.0-0-dev already installed
[OK] dfu-util already installed
[OK] libdbus-1-dev already installed
[OK] usbutils already installed
[OK] libudev-dev already installed

>>> STEP: 2. Configuring udev rules for Bosch Application Board
[sudo] password for rtd3hc:
[OK] udev rules installed
[OK] User 'rtd3hc' is already in dialout group

>>> STEP: 3. Detecting board and flashing coines_bridge firmware
[OK] Detected board: app3.1
dfu-util 0.11
```

Figure 11: Executing setup_bhy2cli.sh

Note: please execute **source ~/.bashrc** to ensure ***PATH*** is updated permanently.
Then, now we can compile BHy2CLI normally by executing: **make**

```
Platform: Linux / macOS
cc:  "/usr/bin/gcc".
[ MKDIR ] build/PC
[ CC ] source/cli.c
[ CC ] source/dinject.c
[ CC ] source/post_mortem.c
[ CC ] source/verbose.c
[ CC ] source/common/common.c
[ CC ] source/callbacks/bhy2cli_callbacks.c
[ CC ] source/callbacks/common_callbacks.c
[ CC ] source/bhy_def/bhy_defs.c
[ CC ] submodules/bhi360/bhi360_activity_param.c
[ CC ] submodules/bhi360/bhi360_api_entry.c
[ CC ] submodules/bhi360/bhi360_bsec_param.c
[ CC ] submodules/bhi360/bhi360_bsx_algo_param.c
[ CC ] submodules/bhi360/bhi360.c
[ CC ] submodules/bhi360/bhi360_event_data.c
[ CC ] submodules/bhi360/bhi360_head_orientation_param.c
[ CC ] submodules/bhi360/bhi360_hif.c
[ CC ] submodules/bhi360/bhi360_logbin.c
[ CC ] submodules/bhi360/bhi360_multi_tap_param.c
[ CC ] submodules/bhi360/bhi360_parse.c
[ CC ] submodules/bhi360/bhi360_phy_sensor_ctrl_param.c
[ CC ] submodules/bhi360/bhi360_system_param.c
[ CC ] submodules/bhi360/bhi360_virtual_sensor_conf_param.c
[ CC ] submodules/bhi360/bhi360_virtual_sensor_info_param.c
[ CC ] submodules/bhi385/source/bhi385_activity_param.c
```

Figure 12: Compiling BHy2CLI in Linux

2.3.4 Running BHy2CLI in MCU mode

In MCU mode, the BHy2CLI application runs in the application board and communicates directly with the BHy Sensor Hub.

Figure 13: MCU Mode

2.3.4.1 Window Environment

Execute BHy2CLI in MCU mode:

1. From release package (Refer figure 4 above), open the folder for the Application board connected (<board>: app30 or app31)
2. For app30,
    a. In ***bin/MCU_APP30***, execute **download_spi.bat** or **download_i2c.bat** script for downloading BHy2CLI binary into MCU
    b. Opening **Hterm** application, and setup something necessary to get output information:

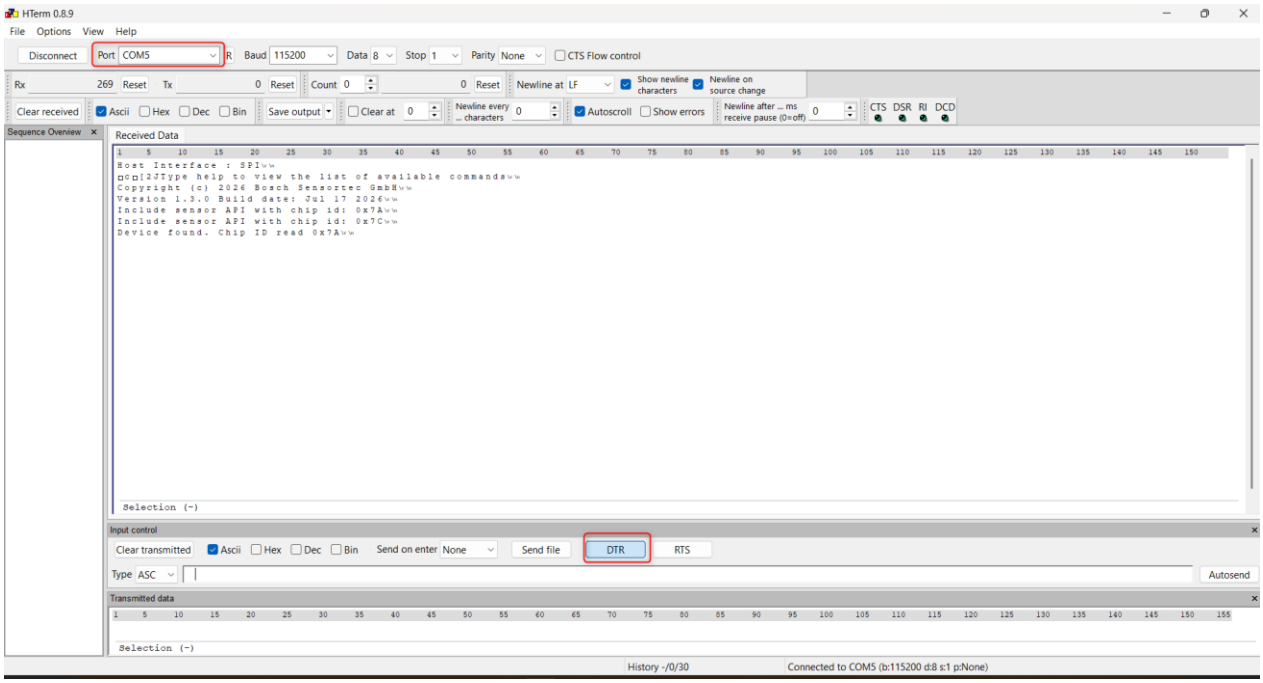

Figure 14: Running BHy2CLI with Hterm in Window

c. If connect BHy2CLI via BLE, opening web-device-cli, and Connect to “bhycli” node

Figure 15: Running BHy2CLI with web-device-cli

3. For app31,
   a. ***In bin/MCU_APP31***, execute **download_spi.bat** or **download_i2c.bat** script for downloading BHy2CLI binary into MCU
   b. For getting output, can do similar as app30 above

Note: The way to check connecting port via USB connection

Platform	***Terminal Application***
Windows	*PuTTY, Hterm etc. Check COMX port in Device Manager*
Linux	*cat command. Eg: cat /dev/ttyACM0*
Mac	***screen** command, eg: screen /dev/tty.usbmodem9F31*

Table 3: Terminal Applications for various OS Environments

2.3.4.2 Linux Environment

Opening terminal applications inside BHy2CLI folder, execute command: **make TARGET=MCU_APP30 download** for Application board 3.0 or **make TARGET=MCU_APP31 download** for Application board 3.1

```
[ AR ] libcoines-mcu_app31.a
[ LD ] bhy2cli.elf
[ BIN ] bhy2cli.bin
[ DFU ] appswitch
dfu-util 0.11

Copyright 2005-2009 Weston Schmidt, Harald Welte and OpenMoko Inc.
Copyright 2010-2021 Tormod Volden and Stefan Schmidt
This program is Free Software and has ABSOLUTELY NO WARRANTY
Please report bugs to http://sourceforge.net/p/dfu-util/tickets/

dfu-util: Warning: Invalid DFU suffix signature
dfu-util: A valid DFU suffix will be required in a future dfu-util release
Opening DFU capable USB device...
Device ID 108c:ab39
Device DFU version 0110
Claiming USB DFU Interface...
Setting Alternate Interface #1 ...
Determining device status...
DFU state(2) = dfuIDLE, status(0) = No error condition is present
DFU mode device DFU version 0110
Device returned transfer size 64
Copying data from PC to DFU device
Download        [=========================] 100%       256680 bytes
Download done.
DFU state(5) = dfuDNLOAD-IDLE, status(0) = No error condition is present
Done!
```

Figure 16: Downloading BHy2CLI into MCU

For USB connection, opening Hterm application and connect to Application board

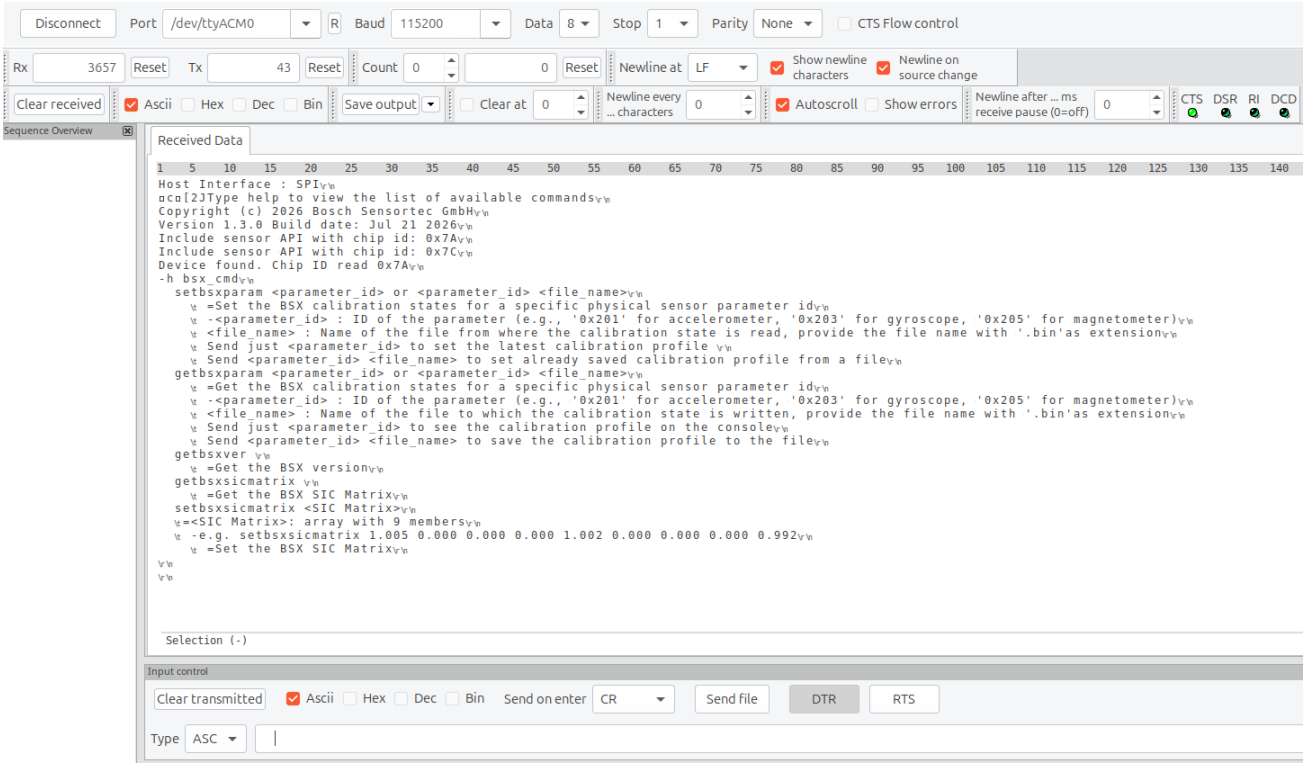

Figure 17: Running BHy2cli with Hterm in Linux

For the BLE connection, opening [web-device-cli] in the Edge or Firefox browser

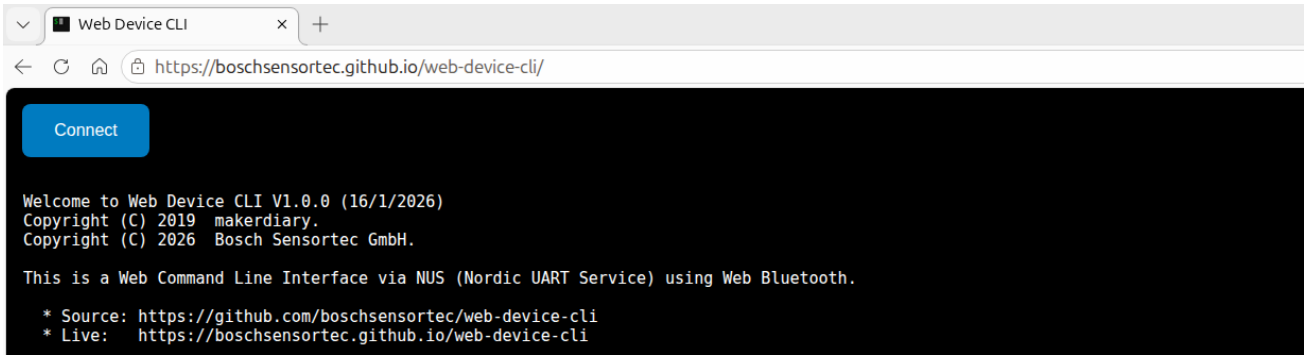

Figure 18: web-device-cli in Edge browser

Note: In the case meet an issue: “WebBluetooth API is not available on your browser”, please open new tab, such as in Edge browser, input: **edge://flags** and enable **Experimental Web Platform features**

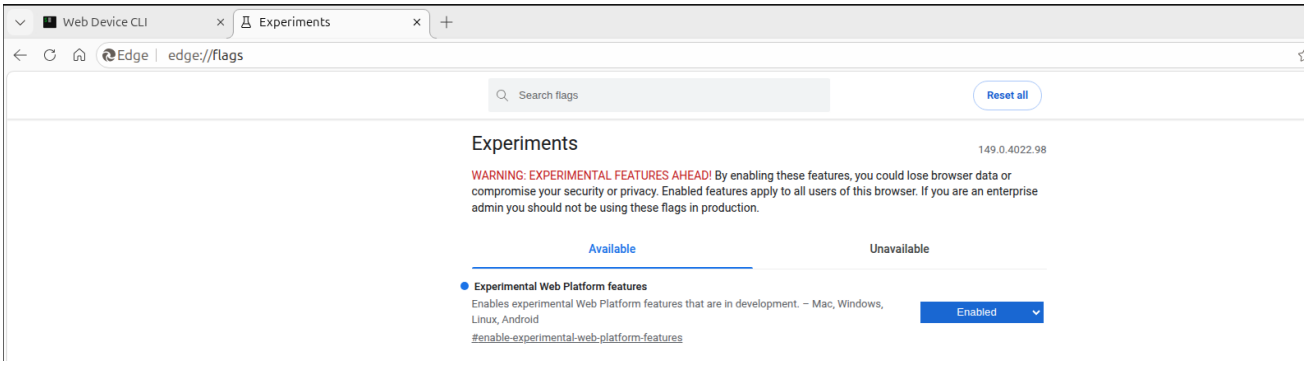

Figure 19: Enabling feature in Edge browser

Furthermore: please execute: **sudo apt-get update** and **sudo apt-get upgrade** to download and install newest available versions of your currently installed software, that will improve the BLE connection.

2.3.5 Running BHy2CLI in PC mode

The BHy2CLI in PC mode is run as a terminal command, and this depends on the local operating system.

Platform	Usage
Windows	spi_bhy2cli.exe [<options>] [<option arguments>] OR spi_ bhy2cli.exe [port] [COMx] [<options>] [<option arguments>]
Linux	./bhy2cli [<options>] [<option arguments>]
MAC	./bhy2cli [<options>] [<option arguments>]

Table 4: PC Mode execution across various OS Environments

2.3.5.1 Window platform

In PC mode, the BHy2CLI application runs on a PC and communicates to the BHy Sensor Hub through a base application running on the application board.
The base application is a middleware between the PC application (BHy2CLI) and the BHy Sensor Hub Firmware (referenced in section 2.3).

Figure 20: PC Mode

Execute BHy2CLI in PC mode:

1. Flashing base firmware to the application board, execute **submodules/coines/firmware/app3.0/coines_bridge/update_coines_bridge_flash_fw.bat** or **submodules/coines/firmware/app3.1/coines_bridge/update_coines_bridge_flash_fw.bat**
2. Reset the board
3. Move to bin/x64 and execute **i2c_bhy2cli.exe** or **spi_bhy2cli.exe**

Note: As the base application is being loaded onto the external Flash of the application board, this step needs to be done only once during the first-time setup.

Note: When running an application in PC mode, each prompt triggers a re-initialization of the application. As such, when multiple BHy2CLI commands need to be executed in sequence, run all the commands in a single command prompt execution, to avoid re-initializing the BHy2CLI each time, e.g.:

spi_BHy2CLI.exe [<cmd_1> <cmd_1_arguments>] [<cmd_2> <cmd_2_arguments>]..[<cmd_n> <cmd_n_arguments>]

Note: When changing from **spi_bhy2cli.exe** to **i2c_bhy2cli.exe** or **i2c_bhy2cli.exe** to **spi_bhy2cli.exe**, it is required to restart the device.

2.3.5.2 Linux Environment

After compiling BHy2CLI at the part 2.3.3.2, we can execute BHy2CLI normally:

```
                                ./bhy2cli -h bsx_cmd
Host Interface : SPI
Copyright (c) 2026 Bosch Sensortec GmbH
Version 1.3.0 Build date: Jul 20 2026
Include sensor API with chip id: 0x7A
Include sensor API with chip id: 0x7C
Device found. Chip ID read 0x7A
  setbsxparam <parameter_id> or <parameter_id> <file_name>
         =Set the BSX calibration states for a specific physical sensor parameter id
         -<parameter_id> : ID of the parameter (e.g., '0x201' for accelerometer, '0x203' for gyroscope, '0x205' for magnetometer)
         <file_name> : Name of the file from where the calibration state is read, provide the file name with '.bin'as extension
         Send just <parameter_id> to set the latest calibration profile
         Send <parameter_id> <file_name> to set already saved calibration profile from a file
  getbsxparam <parameter_id> or <parameter_id> <file_name>
         =Get the BSX calibration states for a specific physical sensor parameter id
         -<parameter_id> : ID of the parameter (e.g., '0x201' for accelerometer, '0x203' for gyroscope, '0x205' for magnetometer)
         <file_name> : Name of the file to which the calibration state is written, provide the file name with '.bin'as extension
         Send just <parameter_id> to see the calibration profile on the console
         Send <parameter_id> <file_name> to save the calibration profile to the file
  getbsxver
         =Get the BSX version
  getbsxsicmatrix
         =Get the BSX SIC Matrix
  setbsxsicmatrix <SIC Matrix>
        =<SIC Matrix>: array with 9 members
         -e.g. setbsxsicmatrix 1.005 0.000 0.000 0.000 1.002 0.000 0.000 0.000 0.992
         =Set the BSX SIC Matrix
```

Figure 21: Executing BHy2CLI in Linux PC

2.4 Sensor Hub Setup

The BHy, as programmable Smart Sensor Hubs, contain an embedded microcontroller that must be loaded with an appropriate firmware image to use their Smart Sensor features.

Figure 22: Smart Sensor Architecture

To load the firmware onto the BHy microcontroller, first select the relevant firmware and use the boot commands of BHy2CLI to load the firmware and boot the device. The BHy generic firmware is in the '**firmwares**' folder of the **Sensor API**.

Note: *User can either use the provided example firmware located in the 'BHI3-firmwares' folder, or they can create a firmware of their own and use it. Details regarding creating a custom firmware can be found in the Bhy FW SDK User Guide*

See ***Boot Commands*** for more details.

3 BHy2CLI Commands

3.1 Overview

BHy2CLI commands are simpler ways to use the deeper BHI360/BHI385 Sensor APIs, the commands supported by the BHy2CLI can be classified as follows:

- Info
- Boot
- Register Read/Write
- Parameter Read/Write
- Data Acquisition
- Log Generation
- Use Case Specific
- System Parameters
- BSX Algorithm Parameters
- Virtual Sensor Information Parameters
- Virtual Sensor Configuration Parameters
- Head Feature Parameters
- Generic Features Parameters
- Physical Sensor Configuration
- Activity Recognition Parameters
- Data Injection
- Diagnostics
- Utility Commands

The following table lists the commands available in BHy2CLI *[v1.3.0]*.

Feature Class	*Command*	*Description*
Info	***-h OR help***	*List the available commands along with their usage*
	version	*Prints the HW, SW and FW version*
	-I OR info	*Show device information*
	-p OR physeninfo	*Display Physical Sensor Information of a physical sensor*
	schema	*Display the schema of the available sensors*
	chipid	*Get Chip ID of the sensor*
Boot	***-n OR reset***	*Reset sensor hub*
	ram	*Upload firmware to RAM*
	-g OR boot	*Boot from the specified medium -ram*
	-b OR ramb	*Reset, upload specified firmware to RAM and boot from RAM*
Register Read/Write	***-r OR rd***	*Read from register*
	-w OR wr	*Write to register*
Parameter Read/Write	***-s OR rdp***	*Read parameter*
	-t OR wrp	*Write parameter*
Data Acquisition	***-c OR actse***	*Activate/Deactivate the sensor in ascii streaming mode*
	hexse	*Activate/Deactivate the sensor in hex streaming mode*
	logse	*Activate/Deactivate the sensor in logging mode*
	dactse	*Deactivate all the active sensors*
	-a OR addse	*Register the expected payload of a new custom virtual sensor*
	lsactse	*List the active sensors and their respective acquisition modes*
Log Generation	***attlog***	*Attach (and create if required) a log file (write-only), where data can be logged to*
	detlog	*Detach the log file*
	logandstream	*Log and stream data for sensor*
Multi Tap	***mtapen***	*Enable the Multi Tap*
	mtapinfo	*Get the Multi Tap Info*
	mtapsetcnfg	*Set the Multi Tap Configurations*

	mtapgetcnfg	*Get the Multi Tap Configurations*
System Parameters	***syssetphyseninfo***	*Set system param physical sensor information*
	sysgetphysenlist	*Get list of physical sensors*
	sysgetvirsenlist	*Get list of virtual sensors*
	sysgettimestamps	*Get system timestamps*
	sysgetfwversion	*Get system firmware version*
	sysgetfifoctrl	*Get FIFO control*
	syssetwkffctrl	*Set watermark for wake-up FIFO control*
	syssetnwkffctrl	*Set watermark for Non wake-up FIFO control*
	sysgetmectrl	*Get meta event control*
	syssetmectrl	*Set meta event control*
	getorientmatrix	*Get the orientation matrix*
	setorientmatrix	*Set the orientation matrix*
BSX Algorithm Parameters	***setbsxparam***	*Set bsx algorithm calibration states*
	getbsxparam	*Get bsx algorithm calibration states*
	getbsxver	*Get the BSX version*
	getbsxsicmatrix	*Get the BSX SIC matrix*
	setbsxsicmatrix	*Set the BSX SIC matrix*
Klio	***kstatus***	*Get Klio status*
	ksetstate	*Set state of Klio*
	kgetstate	*Get state of Klio*
	kreset	*Reset all Klio state*
	kldpatt	*Load Klio pattern for recognition*
	kenpatt	*Enable Klio pattern*
	kdispatt	*Diable Klio pattern*
	kdisapatt	*Disable Klio adaptive pattern*
	kswpatt	*Switch Klio pattern between left/right hand*
	kautldpatt	*Auto-load Klio patterns*
	kgetparam	*Get Klio parameters*
	ksetparam	*Set Klio parameters*
	kgetpattparam	*Get Klio pattern parameters*
	ksetpattparam	*Set Klio pattern parameters*
	ksimscore	*Get Klio Similarity score*
	kmsimscore	*Get Multiple Klio Similarity score*
Virtual Sensor Information Parameters	***virtseinfo***	*Get virtual sensor information*
Virtual Sensor Configuration Parameters	***setvirtsenconf***	*Set virtual sensor configuration*
	getvirtsenconf	*Get virtual sensor configuration*
	accsetfoc	*Set the Accelerometer Fast Offset Calibration*
	accgetfoc	*Get the Accelerometer Fast Offset Calibration*
	accsetpwm	*Set the Accelerometer Power Mode*
	accgetpwm	*Get the Accelerometer Power Mode*
	accsetar	*Set the Accelerometer Axis Remap*
	accgetar	*Get the Accelerometer Axis Remap*
	acctrignvm	*Trigger NVM for Accelerometer*

Physical Sensor Control	***accgetnvm***	*Get the Accelerometer NVM Status*
	gyrosetfoc	*Set the Gyroscope Fast Offset Calibration*
	gyrogetfoc	*Get the Gyroscope Fast Offset Calibration*
	gyrosetois	*Set the Gyroscope OIS state*
	gyrogetois	*Get the Gyroscope OIS status*
	gyrosetfs	*Set the Gyroscope Fast Startup*
	gyrogetfs	*Get the Gyroscope Fast Startup status*
	gyrosetcrt	*Start Gyroscope CRT*
	gyrogetcrt	*Get the Gyroscope CRT status*
	gyrosetpwm	*Set the Gyroscope Power Mode*
	gyrogetpwm	*Get the Gyroscope Power Mode*
	gyrosettat	*Set the Gyroscope Timer Auto Trim state*
	gyrogettat	*Get the Gyroscope Timer Auto Trim status*
	gyrotrignvm	*Trigger NVM for Gyroscope*
	gyrogetnvm	*Get the Gyroscope NVM Status*
	gyrogetmansencomp	*Get the gyroscope manual sensitivity compensation*
	gyrosetmansencomp	*Set the gyroscope manual sensitivity compensation*
	magsetpwm	*Set the Magnetometer Power Mode*
	maggetpwm	*Get the Magnetometer Power Mode*
	wwsetcnfg	*Set the Wrist Wear Wakeup Configuration*
	wwgetcnfg	*Get the Wrist Wear Wakeup Configuration*
	amsetcnfg	*Set the Any Motion Configuration*
	amgetcnfg	*Get the Any Motion Configuration*
	nmsetcnfg	*Set the No Motion Configuration*
	nmgetcnfg	*Get the No Motion Configuration*
	baro1setcnfg	*Set the Barometer Pressure Type 1 Configuration*
	baro1getcnfg	*Get the Barometer Pressure Type 1 Configuration*
	baro2setcnfg	*Set the Barometer Pressure Type 2 Configuration*
	baro2getcnfg	*Get the Barometer Pressure Type 2 Configuration*
	scsetcnfg	*Set the Step Counter Configuration*
	scgetcnfg	*Get the Step Counter Configuration*
	phyrangeconf	*Set the range of physical sensor*
	foc	*Enable the fast offset compensation for the sensor*
	staticcalib	*Execute Accel/Gyro FOC, Gyro CRT*
	getstaticcalib	*Get accel/gyro foc and gyro CRT, store in file*
	setstaticcalib	*Take value from file, set accel/gyro foc and gyro CRT*
Generic Features	***wgdsetcnfg***	*Set the Wrist Gesture Detector Configuration*
	wgdgetcnfg	*Get the Wrist Gesture Detector Configuration*
	wgdresetcnfg	*Reset the Wrist Gesture Detector Configuration*
Activity Recognition Parameters	***sethearactvcnfg***	*Set the Hearable Activity Configuration*
	gethearactvcnfg	*Get the Hearable Activity Configuration*
	setwearactvcnfg	*Set the Wearable Activity Configuration*
	getwearactvcnfg	*Get the Wearable Activity Configuration*
Head Feature	***hmctrig***	*Trigger Head Misalignment Calibration*
	hmcsetcnfg	*Set the Head Misalignment Configuration*
	hmcgetcnfg	*Get the Head Misalignment Configuration*
	hmcsetdefcnfg	*Set the Default Head Misalignment Configuration*
	hmcver	*Get Head Misalignment Calibrator Version*
	hmcsetcalcorrq	*Set the Head Misalignment Quaternion Calibration Correction*
	hmcgetcalcorrq	*Get the Head Misalignment Quaternion Calibration Correction*

	hmcsetmode	*Set the Head Misalignment Mode and Vector X value*
	hmcgetmode	*Get the Head Misalignment Mode and Vector X value*
	hosetheadcorrimu	*Set Initial Heading Correction, only for IMU Head Orientation*
	hogetheadcorrimu	*Get Initial Heading Correction, only for IMU Head Orientation*
	hover	*Get IMU/NDOF Head Orientation Version*
	hosetheadcorrndof	*Set Initial Heading Correction, only for Ndof*
	hogetheadcorrndof	*Get Initial Heading Correction, only for Ndof*
	hgdgetver	*Get Head Gesture Detection version*
	hgdgettimeges	*Get Head Gesture Detection time duration of one gesture*
	hgdsettimeges	*Set Head Gesture Detection time duration of one gesture*
	hgdsetdefault	*Set Head Gesture Detection to default setting*
	hgdgetthresangrat	*Get Head Gesture Detection angular rate threshold*
	hgdsetthresangrat	*Set Head Gesture Detection angular rate threshold*
	hgdgettimegestilt	*Get Head Gesture Detection time duration of tilt gesture*
	hgdsettimegestilt	*Set Head Gesture Detection time duration of tilt gesture*
Data Injection	***dmode***	*Set the Data Injection mode*
	dinject	*Compute virtual sensor output from raw IMU data*
Diagnostics	***-m OR postm***	*Get Post Mortem Data and log to a file*
Utility	***-v OR verb***	*Set the verbose level.*
	echo	*Enable/Disable echo*
	heart	*Enable/Disable Heartbeat Message*
	mklog	*Create a log file*
	rm	*Remove a log file*
	ls	*List the files in the external Flash*
	wrfile	*Write to a log file*
	rdfile	*Read from a log file*
	slabel	*Set a string label in the log file*
	cls	*Clear Screen*

Table 5: BHy2CLI Commands

This chapter explains the usage of the commands with the help of examples to guide users on how to operate the BHy.

Note: *The outputs provided for reference are generated in* ***MCU*** *mode over* ***BLE*** *communication.*

3.2 Info Commands

The Info commands are commands oriented towards giving System and Application specific information.

Feature Class	***Command***	***Description***
Info	***-h OR help***	*List the available commands along with their usage*
	version	*Prints the HW, SW and FW version*
	-i OR info	*Show device information*
	-p OR physeninfo	*Display Physical Sensor Information of a physical sensor*
	schema	*Display the schema of the available sensors*
	chipid	*Get Chip ID of the sensor*

Table 6: Overview of Info Commands

Get Info -	
Action	***Usage***
Get BHy2CLI Application Info	*help*
Get BHy2CLI Application Version Details	*version*
Get System Info	*info*
Get Physical Sensor Information of a physical sensor	*physeninfo <phy_sensor_id>, eg: physeninfo 1*
Get schema of the available sensors	*schema*

Get Chip ID of the sensor	*chipid*

Table 7: Info Commands Usage

3.2.1 Help

Available commands and connection status of the device will be displayed. Usage: **./spi_bhy2cli.exe**

```
help
Usage:
bhy2cli [<port>] [<port_name>] [<options>]
<port>: optional input parameter, trigger keyword
<port_name>: optional input parameter, use it when want to support running multiple applications in parallel
Options:
  -h [<options>] OR help [<options>]
        = Print this usage message
     <options>: optional input parameter, use it to specify the options
         act_cmd          - List of activity param command
         base_cmd         - List of basic command
         bsx_cmd          - List of BSX param command
         gen_cmd          - List of generic features command
         hf_cmd           - List of head feature command
         klio_cmd         - List of klio param command
         mtap_cmd         - List of multitap param command
         phy_ctrl_cmd     - List of physical sensor control param command
         sys_cmd          - List of system param command
         virt_cmd         - List of virtual sensor param command
  version
        = Prints the version
  -v OR verb <verbose level>
        = Set the verbose level. 0 Error, 1 Warning, 2 Infos
  -b OR ramb <firmware path>
        = Reset, upload specified firmware to RAM and boot from RAM
          [equivalent to using "reset ram <firmware> boot r" successively]
  -n OR reset
        = Reset sensor hub
  -a OR addse <sensor id>:<sensor name>:<total output payload in bytes>:
     <output_format_0>:<output_format_1>
        = Register the expected payload of a new custom virtual sensor
         -Valid output_formats:
                 u8:  Unsigned 8 Bit
                 u16: Unsigned 16 Bit
                 u24: Unsigned 24 Bit
                 u32: Unsigned 32 Bit
                 s8:  Signed 8 Bit
                 s16: Signed 16 Bit
                 s24: Signed 24 Bit
                 s32: Signed 32 Bit
                 f:   Float
                 c:   Char
```

Figure 23: 'help' output

In the case want to list of all available command for specific parameters, can use the extension of this command, example:

```
getbsxparam <parameter_id> or <parameter_id> <file_name>
       =Get the BSX calibration states for a specific physical sensor parameter id
       -<parameter_id> : ID of the parameter (e.g., '0x201' for accelerometer, '0x203' for gyroscope, '0x205' for magnetometer)
       <file_name> : Name of the file to which the calibration state is written, provide the file name with '.bin'as extension
       Send just <parameter_id> to see the calibration profile on the console
       Send <parameter_id> <file_name> to save the calibration profile to the file
getbsxver
       =Get the BSX version
getbsxsicmatrix
       =Get the BSX SIC Matrix
setbsxsicmatrix <SIC Matrix>
      =<SIC Matrix>: array with 9 members
       -e.g. setbsxsicmatrix 1.005 0.000 0.000 0.000 1.002 0.000 0.000 0.000 0.992
       =Set the BSX SIC Matrix
```

Figure 24: 'help bsx_cmd' output

3.2.2 Version

Hardware information, Software version. Usage:

```
version
HW info:: Board: 9, HW ID: 11, Shuttle ID: 179, SW ID: 2303
SW Version: 1.3.0
Build date: Jul 22 2026
```

Figure 25: 'version' Output

3.2.3 Info

Display general device information, virtual sensors present in loaded firmware, boot status register content and error register content. Usage:

```
info
Product ID     : 89
Kernel version : 2380
User version   : 9792
ROM version    : 5166
Power state    : sleeping
Host interface : SPI
Feature status : 0x4a
Boot Status : 0x38: No flash installed. Host interface ready. Firmware verification done.
Virtual sensor list.
Sensor ID |                          Sensor Name |  ID | Ver |  Min rate |  Max rate |
----------+--------------------------------------+-----+-----+-----------+-----------|
        1 |            Accelerometer passthrough | 205 |   1 |    1.5625 |  400.0000 |
        3 |           Accelerometer uncalibrated | 203 |   1 |    1.5625 |  400.0000 |
        4 |              Accelerometer corrected | 241 |   1 |    1.5625 |  400.0000 |
        5 |                 Accelerometer offset | 209 |   1 |    1.0000 |    1.0000 |
        6 |      Accelerometer corrected wake up | 192 |   1 |    1.5625 |  400.0000 |
        7 |   Accelerometer uncalibrated wake up | 204 |   1 |    1.5625 |  400.0000 |
       10 |                Gyroscope passthrough | 207 |   1 |    1.5625 |  400.0000 |
       12 |               Gyroscope uncalibrated | 244 |   1 |    1.5625 |  400.0000 |
       13 |                  Gyroscope corrected | 243 |   1 |    1.5625 |  400.0000 |
```

Figure 26: 'info' Output

3.2.4 Physical Sensor Information

Display physical sensor information. Usage:

```
physeninfo 1
Field Name            hex                   | Value (dec)
-------------------------------------------------------
Physical Sensor ID    01                    | 1
Driver ID             1A                    | 26
Driver Version        01                    | 1
Current Consumption   01                    | 0.100mA
Dynamic Range         0008                  | 8
Flags                 22                    | IRQ status       : Disabled
                                            | Master interface : SPI0
                                            | Power mode       : Power Down
Slave Address         19                    | 25
GPIO Assignment       02                    | 2
Current Rate          00000000              | 0.000Hz
Number of axes        03                    | 3
Orientation Matrix    0100010001            | +1 +0 +0 |
                                            | +0 +1 +0 |
                                            | +0 +0 +1 |
Reserved              00                    | 0
```

Figure 27: 'physeninfo' Output

3.2.5 Schema Information

Schema information is following below format

Display the schema list of all virtual sensor supported by firmware. Usage:

To explore the functionality of schema output command of one virtual sensor id 1 -	
Description	***Schema element***
Sensor Data ID	*1*
Name of the sensor data event	*Accelerometer passthrough*
Event size	*6*
Parse format	*s16,s16,s16*
Axis names	*x,y,z*
Scaling factor	*0.000244140625000*
Intended sample rate	*-1.0*
Sensor Properties	*na (not available)*

Table 8: Schema information format

```
1.2
1: Accelerometer passthrough: 6: s16,s16,s16: x,y,z: 0.000244140625000: -1.0: na
4: Accelerometer corrected: 6: s16,s16,s16: x,y,z: 0.000244140625000: -1.0: na
5: Accelerometer offset: 6: s16,s16,s16: x,y,z: 0.000244140625000: -1.0: na
6: Accelerometer corrected wake up: 6: s16,s16,s16: x,y,z: 0.000244140625000: -1.0: na
10: Gyroscope passthrough: 6: s16,s16,s16: x,y,z: 0.061035156250000: -1.0: na
13: Gyroscope corrected: 6: s16,s16,s16: x,y,z: 0.061035156250000: -1.0: na
14: Gyroscope offset: 6: s16,s16,s16: x,y,z: 0.061035156250000: -1.0: na
15: Gyroscope wake up: 6: s16,s16,s16: x,y,z: 0.061035156250000: -1.0: na
37: Game rotation vector: 10: s16,s16,s16,s16,u16: x,y,z,w,ar: 0.000061035156250: -1.0: na
38: Game rotation vector wake up: 10: s16,s16,s16,s16,u16: x,y,z,w,ar: 0.000061035156250: -1.0: na
136: Low Power Step counter: 4: u32: sc: -1.000000000000000: -1.0: na
137: Low Power Step detector: 0: e: e: -1.000000000000000: -1.0: na
143: Low Power Any motion wake up: 0: e: e: -1.000000000000000: -1.0: na
153: Multi Tap Detector: 1: u8: taps: -1.000000000000000: -1.0: na
154: Activity recognition wake up: 2: u16: a: -1.000000000000000: -1.0: na
156: Low Power Wrist Gesture wake up: 1: u8: wrist_gesture: -1.000000000000000: -1.0: na
158: Low Power Wrist Wear wake up: 0: e: e: -1.000000000000000: -1.0: na
159: Low Power No Motion wake up: 0: e: e: -1.000000000000000: -1.0: na
```

Figure 28: 'schema' Output

3.2.6 Chip ID

Display product specific number. Usage:

```
chipid
CHIP_ID : 0x7a
```

Figure 29: 'chipid' Output

3.3 Boot Commands

The boot commands are used to load and boot the firmware of the Bhy sensor hubs.
The firmware can be loaded by the application board into the Fuser2 RAM and then booted**.**

Feature Class	*Command*	*Description*
Boot	***-n OR reset***	*Reset sensor hub*
	-u OR ram	*Upload firmware to RAM*
	-g OR boot	*Boot from the specified medium -ram*
	-b OR ramb	*Reset, upload specified firmware to RAM and boot from RAM*

Table 9: Overview of Boot Commands

To upload firmware to the sensor hub -		
Action	***Location***	***Usage***
Reset the Sensor Hub		*reset*
Upload firmware	*to RAM*	*ram <ram_firmware_path>, eg: ram Bosch_Shuttle3_BHIxyz.fw*
Boot Sensor Hub	*from RAM*	*boot r*
Alternatively, uploading the firmware and booting can be done using a single command	*from RAM*	*ramb <ram_firmware_path>, eg: ramb Bosch_Shuttle3_BHIxyz.fw*

Table 10: Boot Commands Usage

Note: *For **MCU** mode, the application board addresses the firmware path from the external Flash of the application board. The firmware is pre-loaded to the external Flash of the application board. For **PC mode**, the location of the firmware is passed as path.*

```
reset
Reset successful


ram Bosch_Shuttle3_BHI385_BMM350_BMP58X_BME688_bsxsam_ndof.fw
Uploading 145036 bytes of firmware to RAM
Uploading firmware to RAM successful


boot r
Waiting for firmware verification to complete
Boot Status : 0x38: Host interface ready. Firmware verification done.
[D][META EVENT WAKE UP]; T: 0.409468750; Firmware initialized. Firmware version 5991
[D][META EVENT]; T: 0.409468750; Firmware initialized. Firmware version 5991
Booting from RAM successful
```

Figure 30: RAM Boot using 'ram' and 'boot'

```
ramb Bosch_Shuttle3_BHI385_BMM350_BMP58X_BME688_bsxsam_ndof.fw
Reset successful
Uploading 145036 bytes of firmware to RAM
Uploading firmware to RAM successful
Waiting for firmware verification to complete
Boot Status : 0x38: Host interface ready. Firmware verification done.
[D][META EVENT WAKE UP]; T: 0.409390625; Firmware initialized. Firmware version 5991
[D][META EVENT]; T: 0.409390625; Firmware initialized. Firmware version 5991
Booting from RAM successful


boot r
Waiting for firmware verification to complete
Boot Status : 0x38: Host interface ready. Firmware verification done.
Booting from RAM successful
```

Figure 31: RAM Boot using 'ramb'

Note: *Fuser Core in the Bhy Sensor Hub acts as the intermediary between the Host Application and the Physical Sensor on the Bhy Sensor Hub. The interaction of the Fuser Core is defined by the firmware (Bhy Firmware Image), running in it. As such, it is important to make sure that a firmware is loaded onto the Fuser Core. If not, use the above commands to load the firmware.*

3.4 Register Read/Write Commands

The Read/Write commands allow the user to read/write the BHy. This feature can also be extended to read/write various system and application specific parameters.

Write register address [-w]

This command will write specified values (one byte each) to subsequent addresses. Since more than one byte can be written to register 0x00, specifying it will result in writing to subsequent bytes of the chosen register. Writing more than one byte to the register addresses 0x04 ongoing will result in writing to the specified register, as well as the following registers, since these registers only hold one byte each (address auto increment for addresses >= 0x04). Address and length can be specified as decimal or hexadecimal.
Note: The maximum write length is restricted to 46 bytes for the current version of the tool.

Read register address [-r]

This command will read a specified amount of data from subsequent addresses. Since the Host Channel registers 0x01 – 0x03 hold more than one byte, specifying one of these registers will result in reading subsequent bytes from the chosen register. Reading more than one byte from the register addresses 0x04 ongoing will result in reading from the specified register, as well as the following registers, since these registers hold one byte each (address auto increment for addresses >= 0x04). Address and length can be specified as decimal or hexadecimal.
Note: The maximum read length is restricted to 53 bytes for the current version of the tool.

Feature Class	*Command*	*Description*
Register Read/Write	***-r OR rd***	*Read from register*
	-w OR wr	*Write to register*

Table 11: Register Read/Write Commands Overview

To explore the functionality of the register read/write commands -	
Action	***Usage***
Read from a particular register up to n bytes	*rd <reg_addr>:<len>, eg: rd 0x07:4*
Write from a particular register onwards	*wr <reg_addr>=<val1>,<val2>..., eg: wr 0x07=0x0a,0x0b,0x0c,0x0d*

Validate the write operation by reading back the written registers	*rd 0x07:4*

Table 12: Register Read/Write Commands Usage

```
rd 0x07:4
Register address: Data
----------------------
0x07            : 00
0x08            : 00
0x09            : 00
0x0a            : 00
Read complete


wr 0x07=0x0a,0x0b,0x0c,0x0d
Writing address successful


rd 0x07:4
Register address: Data
----------------------
0x07            : 0a
0x08            : 0b
0x09            : 0c
0x0a            : 0d
Read complete
```

Figure 32: Read and Write from register

3.5 Parameter Read/Write Commands

The parameter interface is used to allow the configuration and query the state of the system and the sensors. The parameters differ from the normal registers because the length of data transfer is pre-determined and access to a particular set of parameters is dependent on the loaded firmware. ***[For more details, refer section 13.3 in*** *BHIxyz Datasheet* ***]***.

Read parameter [-s]

Display the read parameter response of the specified parameter ID, without the parameter ID and the response length (first four bytes of the response). Parameter IDs can be provided as decimal or hexadecimal.

Write parameter [-t]

Display the read parameter response of the specified parameter ID, without the parameter ID and the response length (first four bytes of the response. Parameter IDs can be provided as decimal or hexadecimal.

Feature Class	*Command*	*Description*
Parameter Read/Write	***-s OR rdp***	*Read parameter*
	-t OR wrp	*Write parameter*

Table 13: Parameter Read/Write Commands Overview

To explore the functionality of the parameter read/write commands -	
Action	***Usage***
Read a parameter	*rdp <param_id>, eg: rdp 0x103*
Write to a parameter	*wrp <param_id >=<val1>,<val2>.., eg: wrp 0x103=0x05,0x06*

Validate the write operation by reading back the written parameter	*rdp 0x103*

Table 14: Parameter Read/Write Commands Usage

```
rdp 0x103
Byte hex      dec | Data
-----------------------------------------
0x000000        0 |00 00 00 00 00 48 00 00
0x000008        8 |00 00 00 00 00 48 00 00
0x000010       16 |80 01 00 00
Reading parameter 0x0103 successful


wrp 0x103=0x05,0x06
Writing parameter successful


rdp 0x103
Byte hex      dec | Data
-----------------------------------------
0x000000        0 |05 06 00 00 00 48 00 00
0x000008        8 |00 00 00 00 00 48 00 00
0x000010       16 |80 01 00 00
Reading parameter 0x0103 successful
```

Figure 33: Read and Write from parameter

3.6 Data Acquisition Commands

Data acquisition commands allow the user to configure the ODR, latency of specific virtual sensors as well as to control output data quantity and activation duration. The tool provides provision for streaming and logging the data.

Feature Class	***Command***	***Description***
Data Acquisition	***-c OR actse***	*Activate/Deactivate the sensor in ascii streaming mode*
	hexse	*Activate/Deactivate the sensor in hex streaming mode*
	logse	*Activate/Deactivate the sensor in logging mode*
	dactse	*Deactivate all the active sensors*
	-a OR addse	*Register the expected payload of a new custom virtual sensor*
	lsactse	*List the active sensors and their respective acquisition modes*

Table 15: Data Acquisition Commands Overview

Sensor Acquisition -			
Action	***Configuration***	***Mode***	***Usage***
Reset the Sensor Hub			*reset*
Upload firmware	*to RAM*		*ramb <ram_firmware>, eg: ram Bosch_Shuttle3_BHIxyz.fw*
Enable the sensor acquisition	*single sensor*	*ascii streaming*	*actse <id>:<odr>, eg: actse 4:100*
		hex streaming	*hexse <id>:<odr>, eg: hexse 4:100*
		logging	*logse <id>:<odr>, eg: logse 4:100*
	multiple sensors	*ascii streaming*	*actse <id1>:<odr> actse <id2>:<odr>, eg: actse 4:100 actse 13:50*
		hex streaming	*hexse <id1>:<odr> hexse <id2>:<odr>,*

			eg: hexse 4:100 hexse 13:50
		logging	*logse <id1>:<odr> logse <id2>:<odr>, eg: logse 4:100 logse 13:50*
	latency	*ascii streaming*	*actse <id>:<odr>:<latency>, eg: actse 4:100:1000*
		hex streaming	*hexse <id>:<odr>:<latency>, eg: hexse 4:100:1000*
		logging	*logse <id>:<odr>:<latency>, eg: logse 4:100:1000*
List all the active sensors	*generic*	*all modes*	*lsactse*
Disable the sensor acquisition	*single sensor*	*ascii streaming*	*actse <id>:0, eg: actse 4:0*
		hex streaming	*hexse <id>:0, eg: hexse 4:0*
		logging	*logse <id>:0, eg: logse 4:0*
	multiple sensors	*ascii streaming*	*actse <id1>:0 actse <id2>:0, eg: actse 4:0 actse 13:0*
		hex streaming	*hexse <id1>:0 hexse <id2>:0, eg: hexse 4:0 hexse 13:0*
		logging	*logse <id1>:0 logse <id2>:0, eg: logse 4:0 logse 13:0*
	single Shot	*all modes*	*dactse*
Define parsing format for a new/custom sensor	*generic*	*all modes*	*addse<id>:"name":<payload (in bytes)>:<paring_format> eg: addse 161:"Altitude":4:s32*

Table 16: Data Acquisition Commands Usage

Note: *The usage of the* ***'logse'*** *command requires some pre-requisites, which are discussed in* ***Log Generation Commands*****.**

3.6.1 Activate sensor

3.6.1.1 Activate one virtual sensor

Activate a virtual sensor id 4 with ODR is 100Hz. To stop streaming, press Ctrl + C. Usage:

```
actse 4:100


[D][META EVENT]; T: 1320.715406250; Power mode changed for sensor id 4
[D][META EVENT]; T: 1320.715406250; Sample rate changed for sensor id 4
[D][META EVENT]; T: 1320.715062500; Flush complete for sensor id 4
[D]SID: 4; T: 1320.774625000; x: -0.094971, y: 0.891357, z: -0.475586; acc: 0
[D][META EVENT]; T: 1320.774625000; Accuracy for sensor id 4 changed to 1
[D]SID: 4; T: 1320.784640625; x: -0.095947, y: 0.897949, z: -0.480469; acc: 1
[D]SID: 4; T: 1320.794671875; x: -0.096680, y: 0.895508, z: -0.479980; acc: 1
[D]SID: 4; T: 1320.804687500; x: -0.095703, y: 0.897949, z: -0.478027; acc: 1
[D]SID: 4; T: 1320.814703125; x: -0.096191, y: 0.897949, z: -0.480957; acc: 1
[D]SID: 4; T: 1320.824718750; x: -0.094971, y: 0.897705, z: -0.476807; acc: 1
[D]SID: 4; T: 1320.834734375; x: -0.095703, y: 0.898682, z: -0.479736; acc: 1
[D]SID: 4; T: 1320.844750000; x: -0.096680, y: 0.895752, z: -0.479004; acc: 1
[D]SID: 4; T: 1320.854765625; x: -0.094727, y: 0.896729, z: -0.477783; acc: 1
[D]SID: 4; T: 1320.864796875; x: -0.095703, y: 0.899902, z: -0.478516; acc: 1
[D]SID: 4; T: 1320.874812500; x: -0.097656, y: 0.900635, z: -0.480713; acc: 1
```

Figure 34: Activate one virtual sensor

3.6.1.2 Activate one sensor with downsampling

Activate sensor id 3 with ODR is 50Hz and down sampling to 10Hz. To stop streaming, press Ctrl + C. Usage:

```
actse 3:50::10
Sensor ID: 3, sample rate: 50.000000 Hz, latency: 0 ms


[D][META EVENT]; T: 356.202609375; Flush complete for sensor id 3
[D][META EVENT]; T: 356.203046875; Power mode changed for sensor id 3
[D][META EVENT]; T: 356.203046875; Sample rate changed for sensor id 3
[D]SID: 3; T: 356.357531250; x: 0.203125, y: 0.547363, z: -0.803711; acc: 1
[D]SID: 3; T: 356.457140625; x: 0.202881, y: 0.546143, z: -0.803711; acc: 1
[D]SID: 3; T: 356.556734375; x: 0.204102, y: 0.546143, z: -0.803223; acc: 1
[D]SID: 3; T: 356.656343750; x: 0.204346, y: 0.546143, z: -0.803223; acc: 1
[D]SID: 3; T: 356.755937500; x: 0.204102, y: 0.546631, z: -0.803711; acc: 1
[D]SID: 3; T: 356.855546875; x: 0.203857, y: 0.545898, z: -0.803467; acc: 1
[D]SID: 3; T: 356.955140625; x: 0.203857, y: 0.545898, z: -0.803711; acc: 1
[D]SID: 3; T: 357.054750000; x: 0.202881, y: 0.547607, z: -0.804199; acc: 1
```

Figure 35: Activate sensor with downsampling

3.6.1.3 Activate multiple sensors

To activate multiple virtual sensors id 4 and 13, with same ODR 25Hz, usage:

```
actse 4:25 actse 13:25



[D][META EVENT]; T: 10.640328125; Power mode changed for sensor id 4
[D][META EVENT]; T: 10.640328125; Sample rate changed for sensor id 4
[D][META EVENT]; T: 10.639859375; Flush complete for sensor id 4
[D][META EVENT]; T: 10.658671875; Power mode changed for sensor id 13
[D][META EVENT]; T: 10.658671875; Sample rate changed for sensor id 13
[D][META EVENT]; T: 10.657890625; Flush complete for sensor id 13
[D]SID: 4; T: 10.840546875; x: -0.087402, y: 0.917480, z: 0.518066; acc: 0
[D][META EVENT]; T: 10.840546875; Accuracy for sensor id 4 changed to 0
[D]SID: 13; T: 10.840546875; x: 0.305176, y: 0.061035, z: -0.183105; acc: 0
[D][META EVENT]; T: 10.840546875; Accuracy for sensor id 13 changed to 0
[D]SID: 4; T: 10.880734375; x: -0.088379, y: 0.926025, z: 0.523438; acc: 0
[D]SID: 13; T: 10.880734375; x: 0.915527, y: 0.000000, z: -0.244141; acc: 0
[D]SID: 4; T: 10.920937500; x: -0.087891, y: 0.924072, z: 0.522217; acc: 0
[D]SID: 13; T: 10.920937500; x: 2.868652, y: 0.427246, z: -0.305176; acc: 0
```

Figure 36: Activate multiple sensors

3.6.1.4 Activate sensor in HEX mode

Activate virtual sensor id 1 with ODR is 10Hz in HEX mode, usage:

```
hexse 1:10


[D][META EVENT]; T: 1083.120734375; Flush complete for sensor id 1
[D][META EVENT]; T: 1083.121265625; Power mode changed for sensor id 1
[D][META EVENT]; T: 1083.121265625; Sample rate changed for sensor id 1
[H]010000043b1ae50d3e5002c8008710
[H]010000043b1fa853085602cc00ae10
[H]010000043b246bd5db5502cb00a510
[H]010000043b292f1ba55702c900a510
[H]010000043b2df29e785302ca00a710
[H]010000043b32b6214b5602c800a510
[H]010000043b3779e1275502ca00a610
[H]010000043c00a299fa5502c600a410
[H]010000043c056659d65402c900a210
[H]010000043c0a29dca95402cc00a510
[H]010000043c0eed9c855802cb00a410
[H]010000043c13b15c615302c900a510
```

Figure 37: Activate sensor in HEX streaming mode

3.6.1.5 Activate virtual sensor with latency

Activate sensor id 4 with ODR is 200Hz and latency 1000ms, usage:

```
actse 4:200:1000


[D][META EVENT]; T: 294.314843750; Power mode changed for sensor id 4
[D][META EVENT]; T: 294.314843750; Sample rate changed for sensor id 4
[D][META EVENT]; T: 294.314437500; Flush complete for sensor id 4
[D]SID: 4; T: 294.350781250; x: -0.050781, y: 0.796387, z: 0.623535; acc: 1
[D][META EVENT]; T: 294.350781250; Accuracy for sensor id 4 changed to 1
[D]SID: 4; T: 294.355796875; x: -0.051025, y: 0.801514, z: 0.628174; acc: 1
[D]SID: 4; T: 294.360796875; x: -0.052246, y: 0.801025, z: 0.627441; acc: 1
[D]SID: 4; T: 294.365812500; x: -0.053223, y: 0.802490, z: 0.626953; acc: 1
[D]SID: 4; T: 294.370812500; x: -0.053223, y: 0.802490, z: 0.626953; acc: 1
```

Figure 38: Activate sensor with latency

The parameter "*latency*" can control the activated virtual sensor to output sensor data with a delay. The default unit of this parameter is milliseconds. The parameter is optional, and the default is 0ms if not specified.

3.6.2 List Active sensors

List all activated virtual sensor, usage:

```
actse 50:1


[D][META EVENT]; T: 12.897625000; Power mode changed for sensor id 50
[D][META EVENT]; T: 12.897625000; Sample rate changed for sensor id 50
[D][META EVENT]; T: 12.897203125; Flush complete for sensor id 50

logse 52:1


[D][META EVENT]; T: 18.051250000; Power mode changed for sensor id 52
[D][META EVENT]; T: 18.051250000; Sample rate changed for sensor id 52
[D][META EVENT]; T: 18.050453125; Flush complete for sensor id 52

lsactse
Active Sensors -
SID :  50        ODR : 1.00      R :    1        Acquisition : Streaming
SID :  52        ODR : 1.00      R :    1        Acquisition : Logging
No File attached for Logging
```

Figure 39: List the active sensors

3.6.3 Deactivate virtual sensor

De-activate all activated sensors, usage:

```
actse 50:1


[D][META EVENT]; T: 980.431968750; Flush complete for sensor id 50
[D]SID: 50; T: 982.658843750;
[D]SID: 50; T: 983.106703125;
[D]SID: 50; T: 983.574937500;
[D]SID: 50; T: 984.470421875;
[D]SID: 50; T: 985.203000000;

actse 50:0


[D][META EVENT]; T: 993.326859375; Flush complete for sensor id 50
```

Figure 40: Deactivate sensor using actse

```
lsactse
Active Sensors -
SID :  50        ODR : 1.00      R :    1        Acquisition : Streaming
SID :  52        ODR : 1.00      R :    1        Acquisition : Logging
No File attached for Logging

dactse
Deactivated all the Sensors
[D][META EVENT]; T: 207.311296875; Power mode changed for sensor id 50
[D][META EVENT]; T: 207.311296875; Sample rate changed for sensor id 50
[D][META EVENT]; T: 207.315750000; Power mode changed for sensor id 52
[D][META EVENT]; T: 207.315750000; Sample rate changed for sensor id 52

lsactse
No Active Sensors
No File attached for Logging
```

Figure 41: Deactivate sensor using dactse

3.6.4 Custom/New Virtual Sensor

For a new virtual sensor or for any virtual sensor that is not supported in the BHy2CLI application, it is still possible to use the virtual sensor in BHy2CLI by using the *'addse'* command.
Note: Currently, this command can only work with customized firmware:
Bosch_Shuttle3_BHI385_bsxsam_LDO.fw

Feature Class	*Command*	*Description*
Custom/New Virtual Sensor Support	***addse***	*Extend support for a new/custom virtual sensor, which is not yet supported in the BHy2CLI application*

Table 17: Custom/New Virtual Sensor support Extension

Adding support for Custom/New Virtual Sensor -	
Action	***Usage***
Reset the sensor hub	*reset*
Load the custom firmware with support for custom/new virtual sensor	*ramb <custom_firmware> or <flb <custom_firmware>*
Check if support available for custom/virtual by reading the list of virtual sensors	*info*
Extend the support for the Custom/New virtual sensor in BHy	*addse <sensor_id>:"<sensor_name>":<parse_size>:<parse_format>* *eg: addse160:"LDO ":2:c:c*
Enable the custom/new virtual sensor	*actse <custom_sensor_id>, eg: actse 160:50*

Table 18: Adding support for Custom/New Virtual Sensor

Register the output payload of a custom virtual sensor by providing the Sensor ID, an arbitrary name, the total output payload in bytes and each expected output format.
Note: The registration of a custom sensor only applies for the current function call. To stream sensor data, the –c option must be used.

Possible output formats (with corresponding output payload) are:

- u8 : Unsigned 8 bit (1 byte)
- u16 : Unsigned 16 bit (2 bytes)
- u32 : Unsigned 32 bit (4 bytes)
- s8 : Signed 8 bit (1 byte)
- s16 : Signed 16 bit (2 bytes)
- s24 : Signed 24 bit
- s32 : Signed 32 bit (4 bytes)
- f : Float
- c: Char
- flb: Fixed length binary. Cannot be concatenated with other formats
- fls: Fixed length string. Cannot be concatenated with other formats

Example:

Adding, then activate new sensor id 160, its name is ‘LDO’, parse size is 2 bytes, output format is c:c

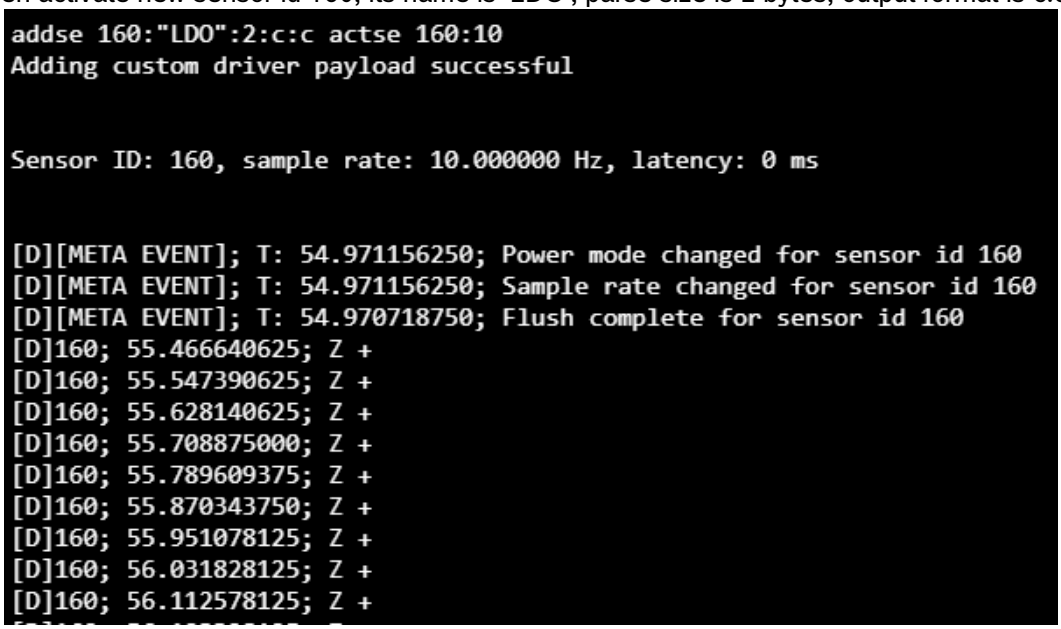

Figure 42: Custom/New Virtual Sensor Detected

```
-i
Product ID     : 89
Kernel version : 2380
User version   : 8224
ROM version    : 5166
Power state    : sleeping
Host interface : SPI
Feature status : 0x4a
Boot Status : 0x38: Host interface ready. Firmware verification done.
Virtual sensor list.
Sensor ID |                          Sensor Name |  ID | Ver |  Min rate |  Max rate |
----------+--------------------------------------+-----+-----+-----------+-----------|
        1 |            Accelerometer passthrough | 205 |   1 |    1.5625 |  400.0000 |
        3 |          Accelerometer uncalibrated | 203 |   1 |    1.5625 |  400.0000 |
        4 |             Accelerometer corrected | 241 |   1 |    1.5625 |  400.0000 |
        5 |                Accelerometer offset | 209 |   1 |    1.5625 |    1.5625 |
        6 |     Accelerometer corrected wake up | 192 |   1 |    1.5625 |  400.0000 |
        7 |   Accelerometer uncalibrated wake up | 204 |   1 |    1.5625 |  400.0000 |
       10 |               Gyroscope passthrough | 207 |   1 |    1.5625 |  400.0000 |
       12 |              Gyroscope uncalibrated | 244 |   1 |    1.5625 |  400.0000 |
       13 |                 Gyroscope corrected | 243 |   1 |    1.5625 |  400.0000 |
       14 |                    Gyroscope offset | 208 |   1 |    1.5625 |    1.5625 |
       15 |                   Gyroscope wake up | 194 |   1 |    1.5625 |  400.0000 |
       16 |      Gyroscope uncalibrated wake up | 195 |   1 |    1.5625 |  400.0000 |
       28 |                      Gravity vector | 247 |   1 |    1.5625 |  400.0000 |
       29 |             Gravity vector wake up | 198 |   1 |    1.5625 |  400.0000 |
       31 |                Linear acceleration | 246 |   1 |    1.5625 |  400.0000 |
       32 |         Linear acceleration wake up | 197 |   1 |    1.5625 |  400.0000 |
       37 |                Game rotation vector | 252 |   1 |    1.5625 |  400.0000 |
       38 |        Game rotation vector wake up | 200 |   1 |    1.5625 |  400.0000 |
       43 |                         Orientation | 254 |   1 |    1.5625 |  400.0000 |
       44 |                Orientation wake up | 202 |   1 |    1.5625 |  400.0000 |
      136 |             Low Power Step counter | 249 |   1 |    1.0000 |    1.0000 |
      137 |            Low Power Step detector | 248 |   1 |    1.0000 |    1.0000 |
      143 |        Low Power Any motion wake up | 191 |   1 |    1.0000 |    1.0000 |
      153 |                 Multi Tap Detector | 182 |   1 |    1.5625 |    1.5625 |
      154 |        Activity recognition wake up | 237 |   1 |    1.5625 |    1.5625 |
      156 |     Low Power Wrist Gesture wake up | 228 |   1 |    1.0000 |    1.0000 |
      158 |        Low Power Wrist Wear wake up | 178 |   1 |    1.0000 |    1.0000 |
      159 |         Low Power No Motion wake up | 181 |   1 |    1.0000 |    1.0000 |
      160 |                               "LDO" | 121 |   1 |    1.5625 |  400.0000 |
```

Figure 43: Custom/New virtual Sensor Acquisition

Note: The new virtual sensor "LDO" is available here

3.7 Log Generation Commands

The log generation commands are used in conjunction with the 'logse' command to acquire the data in logging mode and store the log in the external Flash.

Feature Class	*Command*	*Description*
Log Generation	***attlog***	*Attach (and create if required) a log file (write-only), where data can be logged to*
	detlog	*Detach the log file*
	logandstream	*Log and stream data for sensor*

Table 19: Log Generation Commands Overview

Sensor in Logging mode -

Action	*Usage*
Attach a log file for the logging	*attlog <file_name.file_extension>, eg: attlog abc.bin*
Enable the sensor acquisition in logging mode	*logse <id>:<odr>, eg: logse 4:100*
Disable the sensor acquisition	*logse <id>:0, eg: logse 4:0*
Detach log file from logging	*detlog <file_name.file_extension>, eg: detlog abc.bin*
Log and stream data for sensor with downsampling	*logandstream <sensor id>:<frequency>[:<latency>][:<downsampling>] <filename> <start>* *or logandstream <stop>* *eg: logandstream 3:50::10 4:100::20 log.bin start* *logandstream stop*

Table 20: Log Generation Commands Usage

Example: Create abc.bin file, then store sensor streaming output to this file:

```
attlog abc.bin
File abc.bin was created


logse 4:100


[D][META EVENT]; T: 363.048609375; Flush complete for sensor id 4
[D][META EVENT]; T: 363.049187500; Power mode changed for sensor id 4
[D][META EVENT]; T: 363.049187500; Sample rate changed for sensor id 4
[D][META EVENT]; T: 363.111359375; Accuracy for sensor id 4 changed to 0
[D][META EVENT]; T: 364.246390625; Accuracy for sensor id 4 changed to 1

lsactse
Active Sensors -
SID :   4       ODR : 100.00    R :    8        Acquisition : Logging
Attached Log File : abc.bin


logse 4:0


[D][META EVENT]; T: 393.928468750; Power mode changed for sensor id 4
[D][META EVENT]; T: 393.928468750; Sample rate changed for sensor id 4
[D][META EVENT]; T: 393.928093750; Flush complete for sensor id 4

detlog abc.bin
File abc.bin was detached for logging


lsactse
No Active Sensors
No File attached for Logging
```

Figure 44: Log Generation Output

Example: Display sensor streaming data and store these data in the bin file together:

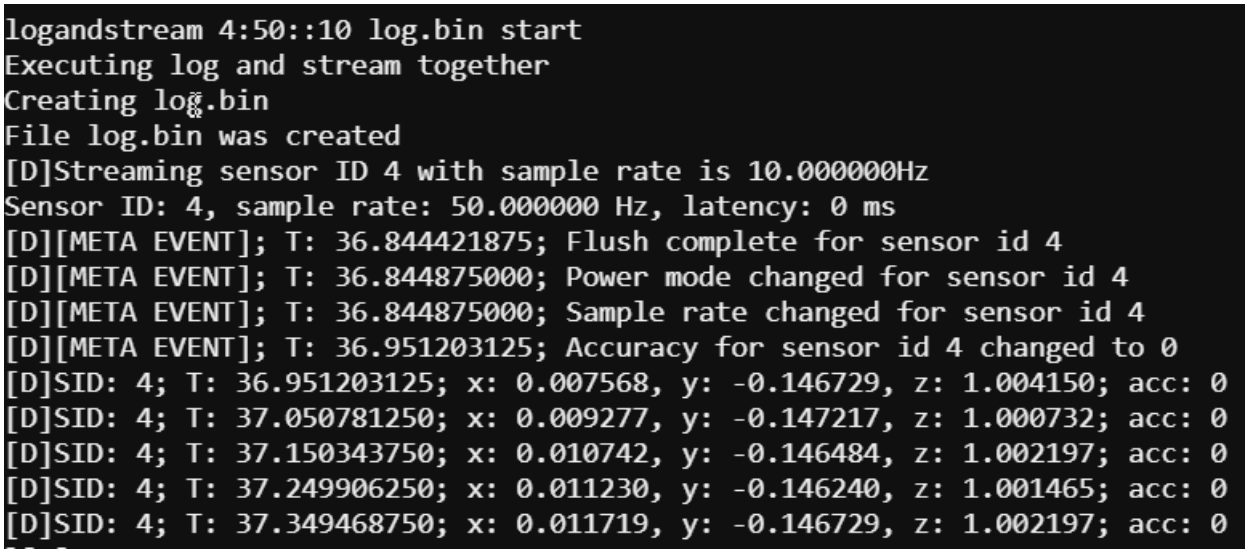

Figure 45: 'logandstream' Command Output

3.8 Use Case Specific Commands

The use case specific commands allow the user to read/write and configure the various controls and parameters for various use case applications such as Multi-tap, etc.

Note: *Before executing various Use Case Specific commands, ensure that firmware for the respective Use Case Applications is flashed onto the BHy Sensor Hub.*

3.8.1 Multi Tap

The multi-tap sensor detects single, double and triple taps. The following commands configure and enable/ disable the various aspects of the multi-tap sensor.

Feature Class	*Command*	*Description*
Multi Tap	***mtapen***	*Enable the Multi Tap*
	mtapinfo	*Get the Multi Tap Info*
	mtapsetcnfg	*Set the Multi Tap Configurations*
	mtapgetcnfg	*Get the Multi Tap Configurations,*

Table 21: Multi-Tap Commands Overview

Multi-Tap commands usage -	
Command	***Usage***
mtapen	*mtapen <tap_setting>, eg: mtapen 2*
mtapinfo	*mtapinfo*
mtapsetcnfg	*mtapsetcnfg <single_tap_config> <double_tap_config> <triple_tap_config>* *eg: mtapsetcnfg 0x0080 0x4022 0x6862* ***Note:*** *The single/double/triple tap configurations passed as arguments to this command are in combined format. Refer above table for specifics. Refer datasheet for bit mapping*
mtapgetcnfg	*mtapgetcnfg*

Table 22: Multi-Tap Commands Usage

Please refer to **Multi-Tap Application Note** for more details on the multi-tap application.

```
mtapinfo
Multi Tap Info : TRIPLE_DOUBLE_SINGLE_TAP

mtapen 2
Multi Tap Parameter set to  DOUBLE_TAP

mtapinfo
Multi Tap Info : DOUBLE_TAP

mtapgetcnfg
Single Tap CNFG : 0x0076
                  -<axis_sel> : 2
                  -<wait_for_timeout> : 1
                  -<max_pks_for_tap> : 6
                  -<mode> : 1
Double Tap CNFG : 0x402d
                  -<tap_peak_thrs> : 45
                  -<max_ges_dur> : 16
Triple Tap CNFG : 0x6864
                  -<max_dur_bw_pks> : 4
                  -<tap_shock_settl_dur> : 6
                  -<min_quite_dur_bw_taps> : 8
                  -<quite_time_after_ges> : 6

mtapsetcnfg 0x0080 0x4022 0x6862
Multi Tap Detector Parameter set successfully

mtapgetcnfg
Single Tap CNFG : 0x0080
                  -<axis_sel> : 0
                  -<wait_for_timeout> : 0
                  -<max_pks_for_tap> : 0
                  -<mode> : 2
Double Tap CNFG : 0x4022
                  -<tap_peak_thrs> : 34
                  -<max_ges_dur> : 16
Triple Tap CNFG : 0x6862
                  -<max_dur_bw_pks> : 2
                  -<tap_shock_settl_dur> : 6
                  -<min_quite_dur_bw_taps> : 8
                  -<quite_time_after_ges> : 6
```

Figure 46: Multi-Tap Commands Output

3.8.2 Head Feature

The head orientation tracking software processes the raw sensor input to estimate head orientation information with respect to the Earth reference system. It combines functional modules and a set of calibration routines to correct the original sensor input, IMU and NDOF fusion algorithms, misalignment estimation between the head and device coordinate system and the rotation transformation between different reference systems.

Feature Class	*Command*	*Description*
	hmctrig	*Trigger Head Misalignment Calibration*

Head Feature	***hmcsetcnfg***	*Set the Head Misalignment Configuration*
	hmcgetcnfg	*Get the Head Misalignment Configuration*
	hmcsetdefcnfg	*Set the Default Head Misalignment Configuration*
	hmcver	*Get Head Misalignment Calibrator Version*
	hmcsetcalcorrq	*Set the Head Misalignment Quaternion Calibration Correction*
	hmcgetcalcorrq	*Get the Head Misalignment Quaternion Calibration Correction*
	hmcsetmode	*Set the Head Misalignment Mode and Vector X value*
	hmcgetmode	*Get the Head Misalignment Mode and Vector X value*
	hosetheadcorrimu	*Set Initial Heading Correction, only for IMU Head Orientation Quaternion*
	hogetheadcorrimu	*Get Initial Heading Correction, only for IMU Head Orientation Quaternion*
	hover	*Get IMU/NDOF Head Orientation Version*
	hosetheadcorrndof	*Set Ndof Initial Heading Correction*
	hogetheadcorrndof	*Get Ndof Initial Heading Correction*
	hgdgetver	*Get the Head Gesture Detection version*
	hgdgettimeges	*Get the Head Gesture Detection time for one gesture*
	hgdsettimeges	*Set the Head Gesture Detection time for one gesture*
	hgdsetdefault	*Set the Head Gesture Detection default configuration*
	hgdgetthresangrat	*Get the Head Gesture Detection angular rate threshold*
	hgdsetthresangrat	*Set the Head Gesture Detection angular rate threshold.*
	hgdgettimegestilt	*Get the Head Gesture Detection time for one tilt gesture*
	hgdsettimegestilt	*Set the Head Gesture Detection time for one tilt gesture, range from 0.1 to 0.3s*

Table 23: Head Feature Commands Overview

Head Orientation commands usage -	
Command	***Usage***
hmcver	*hmcver*
hover	*hover*
hmcsetcnfg	*hmcsetcnfg <sp_max_dur> <sp_min_dur> <dp_max_samples> <acc_diff_th>* *eg: hmcsetcnfg 0x06 0x02 0x32 0x00002042*
hmcgetcnfg	*hmcgetcnfg*
hmcsetdefcnfg	*hmcsetdefcnfg*
hmcsetdefcnfg	*hmcsetdefcnfg*
hmctrig	*hmctrig*
hmcsetcalcorrq	*hmcsetcalcorrq <quat_x> <quat_y> <quat_z> <quat_w>* *eg: hmcsetcalcorrq 1.000 1.000 1.000 1.000*
hmcgetcalcorrq	*hmcgetcalcorrq*
hmcsetmode	*hmcsetmode <mode> <x[0]> <x[1]> <x[2]>* *eg. hmcsetmode 1 0.0 0.0 1.0* ***Note**: The value of Vector X can be changed only in semi mode (<mode> = 1).*
hmcgetmode	*hmcgetmode*
hosetheadcorrimu	*hosetheadcorrimu <enable/disable>* *eg: hosetheadcorrimu 1*
hogetheadcorrimu	*hogetheadcorrimu*
hosetheadcorrndof	*hosetheadcorrndof <enable/disable>* *eg: hosetheadcorrndof 1*
hogetheadcorrndof	*hogetheadcorrndof*
hgdgetver	*hgdgetver*
hgdgettimeges	*hgdgettimeges*
hgdsettimeges	*hgdsettimeges 0.2*
hgdsetdefault	*hgdsetdefault 1*
hgdgetthresangrat	*hgdgetthresangrat*
hgdsetthresangrat	*hgdsetthresangrat 68*
hgdgettimegestilt	*hgdgettimegestilt*

hgdsettimegestilt	*hgdsettimegestilt 0.1*

Table 24: Head Orientation Commands Usage

Please refer to ***Head Orientation Application Note*** for more details on the Head Orientation application.
For hmctrig command, please enable hmc virtual sensor (id 120) before doing head misalignment calibration.

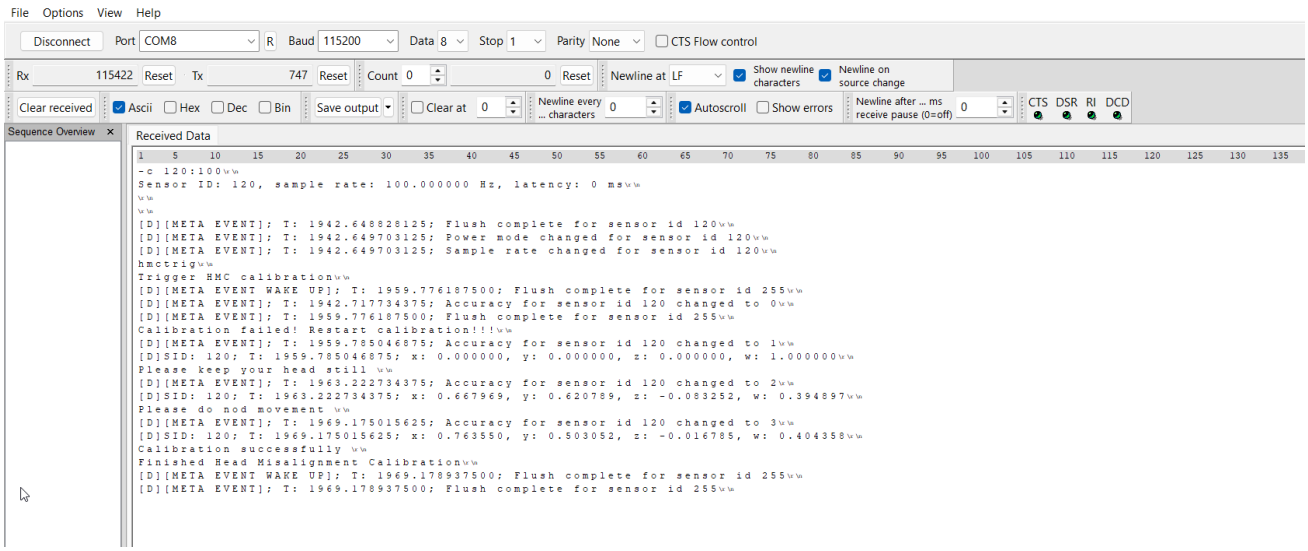

Figure 47: 'hmctrig' output

3.8.3 Cyclic Klio

Klio or Self-Learning AI sensor learns new cyclic gestures and provides responsive recognition of previously learn cyclic gestures together with an instant repetition count. The cyclic gesture needs to be the same in between the repetitions and the BHy needs to come to the original position to close one cycle.

The below commands configure and enable/disable the various aspects of the Klio sensor.

Feature Class	Command	Description
Klio	***kstatus***	*Get Klio status*
	ksetstate	*Set state of Klio*
	kgetstate	*Get state of Klio*
	kreset	*Reset all Klio state*
	kldpatt	*Load Klio pattern for recognition*
	kenpatt	*Enable Klio pattern*
	kdispatt	*Diable Klio pattern*
	kdisapatt	*Disable Klio adaptive pattern*
	kswpatt	*Switch Klio pattern between left/right hand*
	kautldpatt	*Auto-load Klio patterns*
	kgetparam	*Get Klio parameters*
	ksetparam	*Set Klio parameters*

	kgetpattparam	*Get Klio pattern parameters*
	ksetpattparam	*Set Klio pattern parameters*
	ksimscore	*Get Klio Similarity score*
	kmsimscore	*Get Multiple Klio Similarity score*

Table 25: Klio commands overview

Klio commands usage -	
Command	***Usage***
kstatus	*kstatus*
ksetstate	*ksetstate <le> <lr> <re> <rr>* *eg: ksetstate 1 0 0 0*
kgetstate	*kgetstate*
kreset	*kreset*
kldpatt	*kldpatt <index> <pattern>,* *Eg: kldpatt 0* *5242310603a238e7400ad7233c1a8fc640b4e682403db728c1c4826dc0b80b2bc033d2ea3fb2ed* *ddc059feabbf17bf7140555fb23df5aad8bd3482983e8d70c440821e43c03cf5ef402fd901c087548* *3bf16819f3f670446402235e2bf011537c0706bc83de56b64bf16bcf83dea7863bedd3e674021af39* *c020daa4be163f1cbf9f6ace3da79faebf096663c0cd3566c0b0a29ebfd3b3bf3fc8246e3e*
kenpatt	*kenpatt <patterns>* *eg: kenpatt 0*
kdispatt	*kdispatt <patterns>* *eg: kdispatt 0*
kdisapatt	*kdisapatt <patterns>* *eg: kdisapatt 0*
kswpatt	*kswpatt <patterns>* *eg: kswpatt 0*
kautldpatt	*kautldpatt <enable> <index>* *eg: kautldpatt 1 0*
kgetparam	*kgetparam <param>* *eg: kgetparam 2*
ksetparam	*ksetparam <param> <value>* *eg: ksetparam 2 0.5532353542636231*
kgetpattparam	*kgetpattparam <pattern> <param>* *eg: kgetpattparam 2 0*
ksetpattparam	*ksetpattparam <pattern> <param> <value>* *eg: ksetpattparam 2 0 0.9*
ksimscore	*ksimscore <pattern1> <pattern2>* *eg: ksimscore* *5242310603a238e7400ad7233c5513a4c259107a4263f91742151cabc1eacc5cc2015766423dd3* *4d4210529fc23510264242f6bdc1882f2ec2d4404942acbb8741c1b4b7424e4c41bf8d84974182e* *48142e984a042ce05b7426a706cc1779e8c425e156a4136bf5642dc7b51421e0335c13fbc2d42c* *253c4c18ab43942c1942b42071298427829ccc196aa8042d3a345418c9d78c2a6836b40ca1b934* *2* *5242310603a238e7400ad7233c5513a4c259107a4263f91742151cabc1eacc5cc2015766423dd3* *4d4210529fc23510264242f6bdc1882f2ec2d4404942acbb8741c1b4b7424e4c41bf8d84974182e* *48142e984a042ce05b7426a706cc1779e8c425e156a4136bf5642dc7b51421e0335c13fbc2d42c* *253c4c18ab43942c1942b42071298427829ccc196aa8042d3a345418c9d78c2a6836b40ca1b934* *2*

kmsimscore	*kmsimscore <base index> <comparison indices>* *eg: kmsimscore 0 2,4,5*

Table 26: Kilo commands Usage

3.9 System Parameters Commands

The following commands allow the host to set or get system parameters.

Feature Class	Command	Description
System Parameters	***syssetphyseninfo***	*Set system param physical sensor information*
	sysgetphysenlist	*Get list of physical sensors*
	sysgetvirsenlist	*Get list of virtual sensors*
	sysgettimestamps	*Get system timestamps*
	sysgetfwversion	*Get system firmware version*
	sysgetfifoctrl	*Get FIFO control*
	syssetwkffctrl	*Set watermark for wake-up FIFO control*
	syssetnwkffctrl	*Set watermark for Non wake-up FIFO control*
	sysgetmectrl	*Get meta event control*
	syssetmectrl	*Set meta event control*
	getorientmatrix	*Get the orientation matrix*
	setorientmatrix	*Set the orientation matrix*

Table 27: System Parameters Commands Overview

System Parameters commands usage -	
Command	***Usage***
syssetphyseninfo	*syssetphyseninfo <sensor_id> <orientation_matrix>* *eg. syssetphyseninfo 1 0,-1,0,1,0,0,0,0,1*
sysgetphysenlist	*sysgetphysenlist*
sysgetvirsenlist	*sysgetvirsenlist*
sysgettimestamps	*sysgettimestamps*
sysgetfwversion	*sysgetfwversion*
sysgetfifoctrl	*sysgetfifoctrl*
syssetwkffctrl	*syssetwkffctrl <watermark value>* *eg. syssetwkffctrl 500*
syssetnwkffctrl	*syssetnwkffctrl <watermark value>* *eg. syssetnwkffctrl 500*
sysgetmectrl	*sysgetmectrl <address>* *eg. sysgetmectrl 0x101*
syssetmectrl	*syssetmectrl <address> <group idx> <value>* *eg. syssetmectrl 0x101 1 1*
getorientmatrix	*getorientmatrix*
setorientmatrix	*setorientmatrix* eg. *setorientmatrix* 1 -1 0 0 0 -1 0 0 0 -1

Table 28: System Parameters Commands Usage

```
sysgetvirsenlist
Virtual sensor list.
Sensor ID |                              Sensor Name |
----------+------------------------------------------|
        1 |                Accelerometer passthrough |
        3 |              Accelerometer uncalibrated |
        4 |                 Accelerometer corrected |
        5 |                    Accelerometer offset |
        6 |         Accelerometer corrected wake up |
        7 |     Accelerometer uncalibrated wake up |
       10 |                   Gyroscope passthrough |
       12 |                 Gyroscope uncalibrated |
       13 |                    Gyroscope corrected |
       14 |                       Gyroscope offset |
       15 |                     Gyroscope wake up |
       16 |         Gyroscope uncalibrated wake up |
       28 |                          Gravity vector |
       29 |                  Gravity vector wake up |
       31 |                     Linear acceleration |
       32 |             Linear acceleration wake up |
       37 |                    Game rotation vector |
       38 |           Game rotation vector wake up |
       43 |                             Orientation |
       44 |                       Orientation wake up |
      136 |                  Low Power Step counter |
      137 |                Low Power Step detector |
      143 |           Low Power Any motion wake up |
      153 |                     Multi Tap Detector |
      154 |     Activity recognition for Wearables |
      156 |        Low Power Wrist Gesture wake up |
      158 |           Low Power Wrist Wear wake up |
      159 |            Low Power No Motion wake up |
```

```
sysgettimestamps
Host interrupt timestamp: 383.323375000
Current timestamp: 457.535703125
Timestamp event: 0.000000000


sysgetfwversion

Custom version number: 0
EM Hash: 1755aba11aa9
BST Hash: ac49f211
User Hash: ac49f211


sysgetfifoctrl

Wakeup FIFO Watermark = 0
Wakeup FIFO size =  17920
Non Wakeup FIFO Watermark = 0
Non Wakeup FIFO size = 18432
```

```
syssetwkffctrl 500
FIFO wake-up watermark SET Success



syssetnwkffctrl 500
FIFO non wake-up watermark SET Success


syssetmectrl 0x101 1 1
System meta event control SET Success




sysgetmectrl 0x101

Meta event infomation:
0 1 1 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 1
0 1 0 1 0 0 1 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0
```

```
syssetphyseninfo 1 0,-1,0,1,0,0,0,0,1
Set the orientation matrix for the Physical Sensors successfully
sysgetphysenlist

Physical sensor list.
Sensor ID |                          Sensor Name
----------+-------------------------------------|
        1 |                        Accelerometer
        3 |                            Gyroscope
       20 |                 Undefined sensor ID
       32 |                         Step Counter
       33 |                        Step Detector
       35 |                          Any Motion
       52 |                 Activity Recognition
       55 |                           No Motion
       56 |               Wrist Gesture Detector
       57 |                    Wrist Wear Wakeup
```

```
getorientmatrix
Acc 1,0,0,0,1,0,0,0,1

Gyro 1,0,0,0,1,0,0,0,1

setorientmatrix  1 -1 0 0 0 -1 0 0 0 -1
Set the orientation matrix for the Physical Sensors successfully
getorientmatrix
Acc -1,0,0,0,-1,0,0,0,-1

Gyro 1,0,0,0,1,0,0,0,1
```

Figure 48: System Parameters Commands Output

3.10 BSX Algorithm Parameters Commands

The following commands allow the host to set or get BSX Algorithm parameters.

Feature Class	*Command*	*Description*
BSX Algorithm Parameters	***setbsxparam***	*Set bsx algorithm calibration states*
	getbsxparam	*Get bsx algorithm calibration states*
	getbsxver	*Get the BSX version*
	getbsxsicmatrix	*Get BSX SIC matrix*
	setbsxsicmatrix	*Set BSX SIC matrix*

Table 29: BSX Algorithm Parameters Commands Overview

BSX Algorithm commands usage -	
Command	***Usage***
setbsxparam	*setbsxparam <parameter_id> or <parameter_id> <file_name>* *eg. setbsxparam 0x201*
getbsxparam	*getbsxparam <parameter_id> or <parameter_id> <file_name>* *eg. getbsxparam 0x201*
getbsxver	*getbsxver*
getbsxsicmatrix	*getbsxsicmatrix*
setbsxsicmatrix	*setbsxsicmatrix <sic_matrix>*

Table 30: BSX Algorithm Parameters Commands Usage

To get calibration profile of BSX parameter id 0x201, usage:

```
getbsxparam 0x201


Get Calibration profile of BSX parameter id: 0x0201
Block number: 0
Completion flag: 1, Transfer complete
Block length: 25
Struct length: 25
-----------------------------------------
Block data
Byte Hex      Dec | Data
-----------------------------------------
0x000000        0 |00 00 00 00 00 00 00 00
0x000008        8 |00 00 00 00 00 00 00 80
0x000010       16 |3f 00 00 80 3f 00 00 80
0x000018       24 |3f


getbsxparam 0x201 acc.bin


Get Calibration profile of BSX parameter id: 0x0201

Calibration profile for BSX parameter id 0x0201 is saved to the file acc.bin
```

```
setbsxparam 0x201


Setting Calibration profile of BSX parameter id 0x0201 is completed
```

To read calibration profile from file and set to BSX parameter id 0x201, usage:

```
setbsxparam 0x201 acc.bin

Calibration profile for BSX parameter id 0x0201 is read from the file acc.bin and calibrated
```

To get BSX SIC matrix and set BSX matrix with new values, usage:

```
getbsxsicmatrix
BSX SIC Matrix:
1.00000000 0.00000000 0.00000000 0.00000000 1.00000000 0.00000000 0.00000000 0.00000000 1.00000000

setbsxsicmatrix 0.908583 0.0187744 0.0688107 0.0187744 0.858548 0.0496729 0.0688107 0.0496729 0.91861
Setting BSX SIC Matrix successfully
```

To get BSX version, usage:

```
getbsxver
BSX version: 4.0.84.3
```

Figure 49: BSX Algorithm Parameters Commands Output

3.11 Virtual Sensor Information Parameters Commands

The following command allows the host to get virtual sensor information parameters.

Feature Class	*Command*	*Description*
Virtual Sensor Information Parameters	***virtseinfo***	*Get virtual sensor information*

Table 31: Virtual Sensor Information Parameters Command Overview

Virtual Sensor Information Parameters commands usage -	
Command	***Usage***
virtseinfo	*virtseinfo <sensor_id>*

Table 32: Virtual Sensor Information Parameters Command Usage

To get sensor information of virtual sensor id 1, usage:

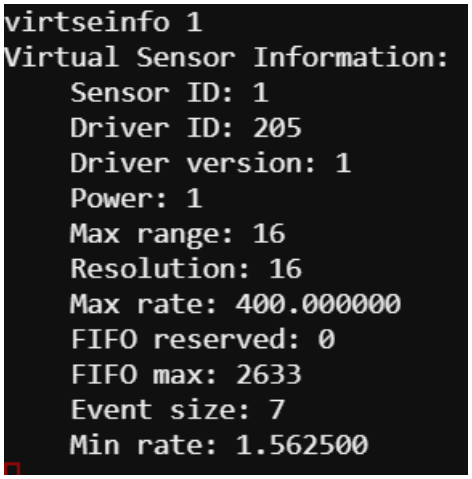

Figure 50: Virtual Sensor Information Command Output

3.12 Virtual Sensor Configuration Parameters Commands

The following commands allow the host to set or get Virtual Sensor Configuration Parameters.

Feature Class	Command	Description
Virtual Sensor Configuration Parameters	***setvirtsenconf***	*Set virtual sensor configuration*
	getvirtsenconf	*Get virtual sensor configuration*

Table 33: Virtual Sensor Configuration Parameters Commands Overview

Virtual Sensor Configuration Parameters commands usage -	
Command	***Usage***
setvirtsenconf	*setvirtsenconf <sensor_id> <sample_rate> <latency>* *eg. setvirtsenconf 3 10 0*
getvirtsenconf	*getvirtsenconf <sensor_id>*

Table 34: Virtual Sensor Configuration Parameters Commands Usage

To set virtual sensor configuration for virtual sensor id 3, sample rate 10Hz and latency is 0ms, usage:

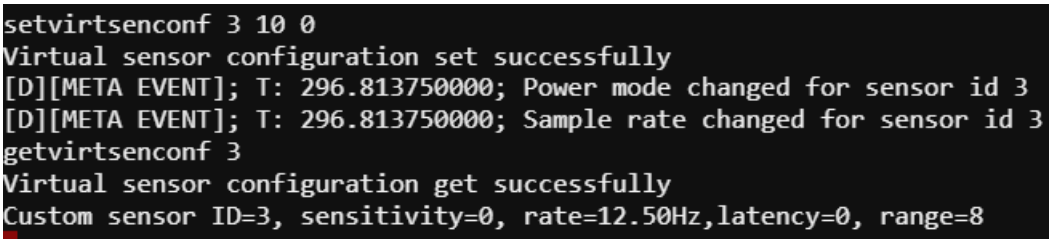

Figure 51: Virtual Sensor Configuration Parameters Commands

3.13 Physical Sensor Configuration Commands

The following commands allow the host to set or get sensor control information of the physical sensors.

Feature Class	Command	Description
Physical Sensor Control	***accsetfoc***	*Set the Accelerometer Fast Offset Calibration*
	accgetfoc	*Get the Accelerometer Fast Offset Calibration*
	accsetpwm	*Set the Accelerometer Power Mode*
	accgetpwm	*Get the Accelerometer Power Mode*
	accsetar	*Set the Accelerometer Axis Remap*
	accgetar	*Get the Accelerometer Axis Remap*

	acctrignvm	*Accelerometer Trigger NVM*
	accgetnvm	*Get the Accelerometer NVM Status*
	gyrosetfoc	*Set the Gyroscope Fast Offset Calibration*
	gyrogetfoc	*Get the Gyroscope Fast Offset Calibration*
	gyrosetois	*Set the Gyroscope OIS state*
	gyrogetois	*Get the Gyroscope OIS status*
	gyrosetfs	*Set the Gyroscope Fast Startup*
	gyrogetfs	*Get the Gyroscope Fast Startup status*
	gyrosetcrt	*Start Gyroscope CRT*
	gyrogetcrt	*Get the Gyroscope CRT status*
	gyrosetpwm	*Set the Gyroscope Power Mode*
	gyrogetpwm	*Get the Gyroscope Power Mode*
	gyrosettat	*Set the Gyroscope Timer Auto Trim state*
	gyrogettat	*Get the Gyroscope Timer Auto Trim status*
	gyrotrignvm	*Gyroscope Trigger NVM*
	gyrogetnvm	*Get the Gyroscope NVM Status*
	gyrosetmansencomp	*Set the Gyroscope manual sensitivity compensation*
	gyrogetmansencomp	*Get the Gyroscope manual sensitivity compensation*
	magsetpwm	*Set the Magnetometer Power Mode*
	maggetpwm	*Get the Magnetometer Power Mode*
	wwwsetcnfg	*Set the Wrist Wear Wakeup Configuration*
	wwwgetcnfg	*Get the Wrist Wear Wakeup Configuration,*
	amsetcnfg	*Set the Any Motion*
	amgetcnfg	*Get the Any Motion Configuration*
	nmsetcnfg	*Set the No Motion Configuration*
	nmgetcnfg	*Get the No Motion Configuration*
	wgdsetcnfg	*Set the Wrist Gesture Detector Configuration*
	wgdgetcnfg	*Get the Wrist Gesture Detector Configuration*
	baro1setcnfg	*Set the Barometer Pressure Type 1 Configuration*
	baro1getcnfg	*Get the Barometer Pressure Type 1 Configuration*
	baro2setcnfg	*Set the Barometer Pressure Type 2 Configuration*
	baro2getcnfg	*Get the Barometer Pressure Type 2 Configuration*
	scsetcnfg	*Set the Step Counter Configuration*
	scgetcnfg	*Get the Step Counter Configuration*
	phyrangeconf	*Set the range of physical sensor*
	foc	*Enable the fast offset compensation for the sensor*
	staticcalib	*Execute Accel/Gyro FOC, Gyro CRT*
	getstaticcalib	*Get accel/gyro foc and gyro CRT, store in file*
	setstaticcalib	*Take value from file, set accel/gyro foc and gyro CRT*

Table 35: Physical Sensor Control Parameter Commands Overview

3.13.1 Accelerometer

The following commands allow the user to configure the accelerometer sensor.

Feature Class	***Command***	***Description***
Accelerometer Sensor Control	***accsetfoc***	*Set the Accelerometer Fast Offset Calibration*
	accgetfoc	*Get the Accelerometer Fast Offset Calibration*
	accsetpwm	*Set the Accelerometer Power Mode*
	accgetpwm	*Get the Accelerometer Power Mode*
	accsetar	*Set the Accelerometer Axis Remap*

	accgetar	*Get the Accelerometer Axis Remap*
	acctrignvm	*Trigger NVM for Accelerometer*
	accgetnvm	*Get the Accelerometer NVM Status*

Table 36: Accelerometer Control Parameter Commands Overview

Parameter	Description
Fast Offset Calibration	*FOC is a one-shot process that compensates errors of the sensor by setting the compensation registers to the negated offset error*
Power Mode	*The power mode of the accelerometer can be configured to operate in other power modes in order to facilitate use case appropriate performance. The accelerometer sensor has 2 power modes –* *0 - Normal Mode, The default state of the sensor* *2 - Low Power, Low power mode on the account of trade-of of power consumption versus sensor noise. Low power mode may introduce aliasing artifacts*
Axis remapping for internal imu features	*The axis remapping function only works for the features in BHy, and it would not influence the IMU outputs (accel data and gyro data).* *It can change the sign for each axis and switch between each axis.*
Triggering a NVM writing	*Once NVM writing process is triggered, the value in backup registers value(offset/crt) will be written into NVM area.*

Table 37: Accelerometer Control Parameters

Accelerometer Parameter commands usage -	
Command	***Usage***
accsetfoc	*accsetfoc <x_offset> <y_offset> <z_offset>, eg: accsetfoc 0x0016 0x00f8 0x07f*
accgetfoc	*accgetfoc*
accsetpwm	*accsetpwm <power_mode>, eg: accsetpwm 2*
accgetpwm	*accgetpwm*
accsetar	*accsetar <x> <x_sign> <y> <y_sign> <z> <z_sign>* *eg: accsetar 0 1 1 1 2 1*
accgetar	*accgetar*
acctrignvm	*acctrignvm*
accgetnvm	*accgetnvm*

Table 38: Accelerometer Control Parameter Commands Usage

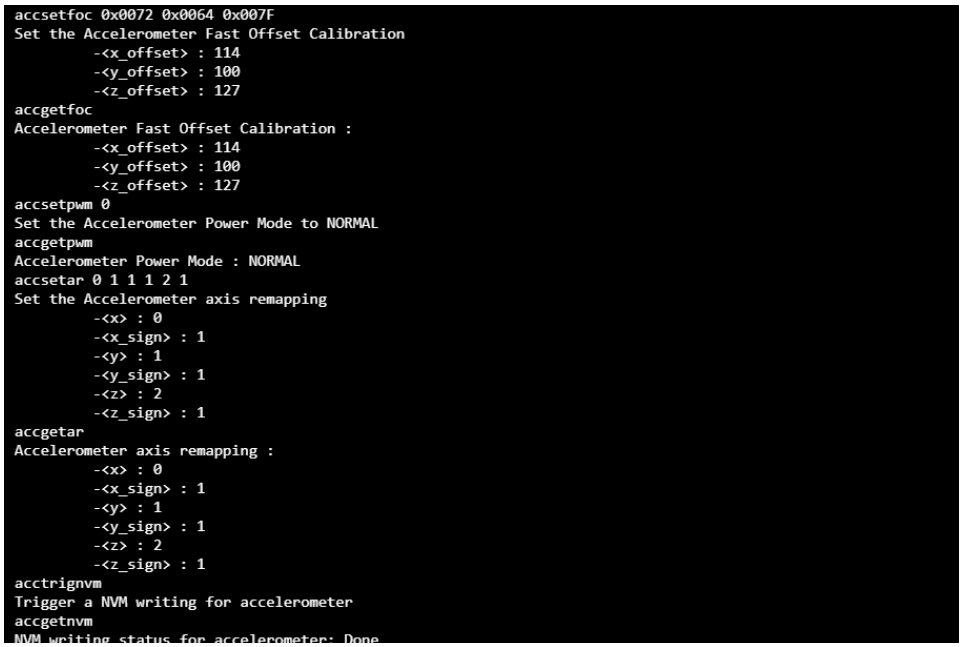

Figure 52: Accelerometer Control Parameter Commands Output

3.13.2 Gyroscope

The following commands allow the user to configure the gyroscope sensor.

Feature Class	Command	Description
Gyroscope Sensor Control	***gyrosetfoc***	*Set the Gyroscope Fast Offset Calibration*
	gyrogetfoc	*Get the Gyroscope Fast Offset Calibration*
	gyrosetois	*Set the Gyroscope OIS state*
	gyrogetois	*Get the Gyroscope OIS status*
	gyrosetfs	*Set the Gyroscope Fast Startup*
	gyrogetfs	*Get the Gyroscope Fast Startup status*
	gyrosetcrt	*Start Gyroscope CRT*
	gyrogetcrt	*Get the Gyroscope CRT status*
	gyrosetpwm	*Set the Gyroscope Power Mode*
	gyrogetpwm	*Get the Gyroscope Power Mode*
	gyrosettat	*Set the Gyroscope Timer Auto Trim state*
	gyrogettat	*Get the Gyroscope Timer Auto Trim status*
	gyrotrignvm	*Trigger NVM for Gyroscope*
	gyrogetnvm	*Get the Gyroscope NVM Status*
	gyrogetmansencomp	*Get the Gyroscope manual sensitivity compensation*
	gyrosetmansencomp	*Set the Gyroscope manual sensitivity compensation*

Table 39: Gyroscope Control Parameter Commands Overview

Parameter	Description
Fast Offset Calibration	*FOC is a one-shot process that compensates errors of the sensor by setting compensation registers to the negated offset error*
Optical Image Stabilization	*Optical Image Stabilization mode of the gyroscope sensor*
Fast Startup Mode	*Fast Startup Mode, when enabled, powers down the measurement part of the Sensor frontend, while keeping the drive and digital parts operational. This allows a faster transition into normal mode while keeping power consumption significantly lower than in normal mode.*
Component Re-Trim	*Trigger and read back the status of the Component Re-Trimming of the gyroscope sensor.*
Power Mode	*The power mode of the gyroscope can be configured to operate in other power modes in order to facilitate use case appropriate performance. The gyroscope sensor has 3 power modes –* *0 - Normal Mode, The default state of the sensor* *1 - Performance Mode* *2 - Low Power, Low power mode on the account of trade-of of power consumption versus sensor noise. Low power mode may introduce aliasing artifacts*
Timer Auto Trim	*Timer Auto Trim allows to sync the Fuser2 timer oscillator PLL. When enabled, the frequency of the timer oscillator is referenced by gyroscope sample rate, benefitting from the high stability of the gyroscope MEMS oscillator. This feature is only applicable when gyroscope is enabled.*
Triggering a NVM writing	*Once NVM writing process is triggered, the value in backup registers value(offset/crt) will be written into NVM area.*
Manual sensitivity compensation	*Write the ratio of reference system report and gyro report into the gyro gain update registers.*

Table 40: Gyroscope Control Parameters

Gyroscope Parameter commands usage -	
Command	***Usage***
gyrosetfoc	*gyrosetfoc <x_offset> <y_offset> <z_offset>, eg: gyrosetfoc 0x0016 0x00f8 0x0080*
gyrogetfoc	*gyrogetfoc*

gyrosetois	*gyrosetois <enable/disable>, eg: gyrosetois 1*
gyrogetois	*gyrogetois*
gyrosetfs	*gyrosetfs <enable/disable>, eg: gyrosetfs 1*
gyrogetfs	*gyrogetfs*
gyrosetcrt	*gyrosetcrt*
gyrogetcrt	*gyrogetcrt*
gyrosetpwm	*gyrosetpwm <power_mode>, eg: gyrosetpwm 2*
gyrogetpwm	*gyrogetpwm*
gyrosettat	*gyrosettat <enable/disable>, eg: gyrosettat 1*
gyrogettat	*gyrogettat*
gyrotrignvm	*gyrotrignvm*
gyrogetnvm	*gyrogetnvm*
gyrogetmansencomp	*gyrogetmansencomp*
gyrosetmansencomp	*gyrosetmansencomp <ratio_x> <ratio_y><ratio_z>, eg: gyrosetmansencomp 0x300 0x400 0x500*

Table 41: Gyroscope Control Parameter Commands Usage

```
gyrosetfoc 0x0016 0x00f8 0x0080
Set the Gyroscope Fast Offset Calibration
        -<x_offset> : 22
        -<y_offset> : 248
        -<z_offset> : 128
gyrogetfoc
Gyroscope Fast Offset Calibration :
        -<x_offset> : 22
        -<y_offset> : 248
        -<z_offset> : 128
gyrosetois 1
Gyroscope OIS Enabled
gyrogetois
Gyroscope OIS Status : Enabled
gyrosetfs 1
Gyroscope Fast Startup Enabled
gyrogetfs
Gyroscope Fast Startup Status : Enabled
gyrosetcrt
Start Gyroscope Component ReTrim (CRT)
gyrogetcrt
Gyroscope CRT Status : Successful
gyrosetpwm 2
Set the Gyroscope Power Mode to LOW POWER
gyrogetpwm
Gyroscope Power Mode : LOW POWER
gyrosettat 1
Gyroscope Timer Auto Trim Started
gyrogettat
Gyroscope Timer Auto Trim Status : Started
gyrotrignvm
Trigger a NVM writing for gyroscope
gyrogetnvm
NVM writing status for gyroscope: Done
```

Figure 53: Gyroscope Control Parameter Commands Output

3.13.3 Magnetometer

The following commands allow the user to configure the Magnetometer sensor.

Feature Class	Command	Description
Magnetometer Sensor Control	***magsetpwm***	*Set the Magnetometer Power Mode*
	maggetpwm	*Get the Magnetometer Power Mode*

Table 42: Magnetometer Control Parameter Commands Overview

Parameter	Description
Power Mode	*The power mode of the magnetometer can be configured to operate in other power modes in order to facilitate use case appropriate performance. The magnetometer sensor has 2 power modes –* *0 - Normal Mode, The default state of the sensor* *2 - Low Power, Low power mode on the account of trade-of of power consumption versus sensor noise. Low power mode may introduce aliasing artifacts*

Table 43: Magnetometer Control Parameters

Magnetometer Parameter commands usage -	
Command	***Usage***
magsetpwm	*magsetpwm <power_mode>* *eg: magsetpwm 0*
maggetpwm	*maggetpwm*

Table 44: Magnetometer Control Parameter Commands Usage

```
magsetpwm 0
Set the Magnetometer Power Mode to NORMAL
maggetpwm
Magnetometer Power Mode : NORMAL
```

Figure 54: Magnetometer Control Parameter Commands Output

3.13.4 Wrist Wear Wakeup

The following commands allow the user to configure the Wrist Wear Wakeup sensor.

Feature Class	Command	Description
Wrist Wear Wakeup Sensor Control	***wwwsetcnfg***	*Set the Wrist Wear Wakeup Configuration*
	wwwgetcnfg	*Get the Wrist Wear Wakeup Configuration,*

Table 45: Wrist Wear Wakeup Control Parameter Commands Overview

Wrist Wear Wakeup Parameter commands usage -	
Command	***Usage***
wwwsetcnfg	*wwwsetcnfg <maf> <manf> <alr> <all> <apd> <apu> <mdm> <mdq>, <alrd> <alld> <apdd> <apud> <maczd>* *eg: wwwsetcnfg 1700 1600 120 100 150 225 5 7 237 237 150 250 100*
wwwgetcnfg	*wwwgetcnfg*

Table 46: Wrist Wear Wakeup Control Parameter Commands Usage

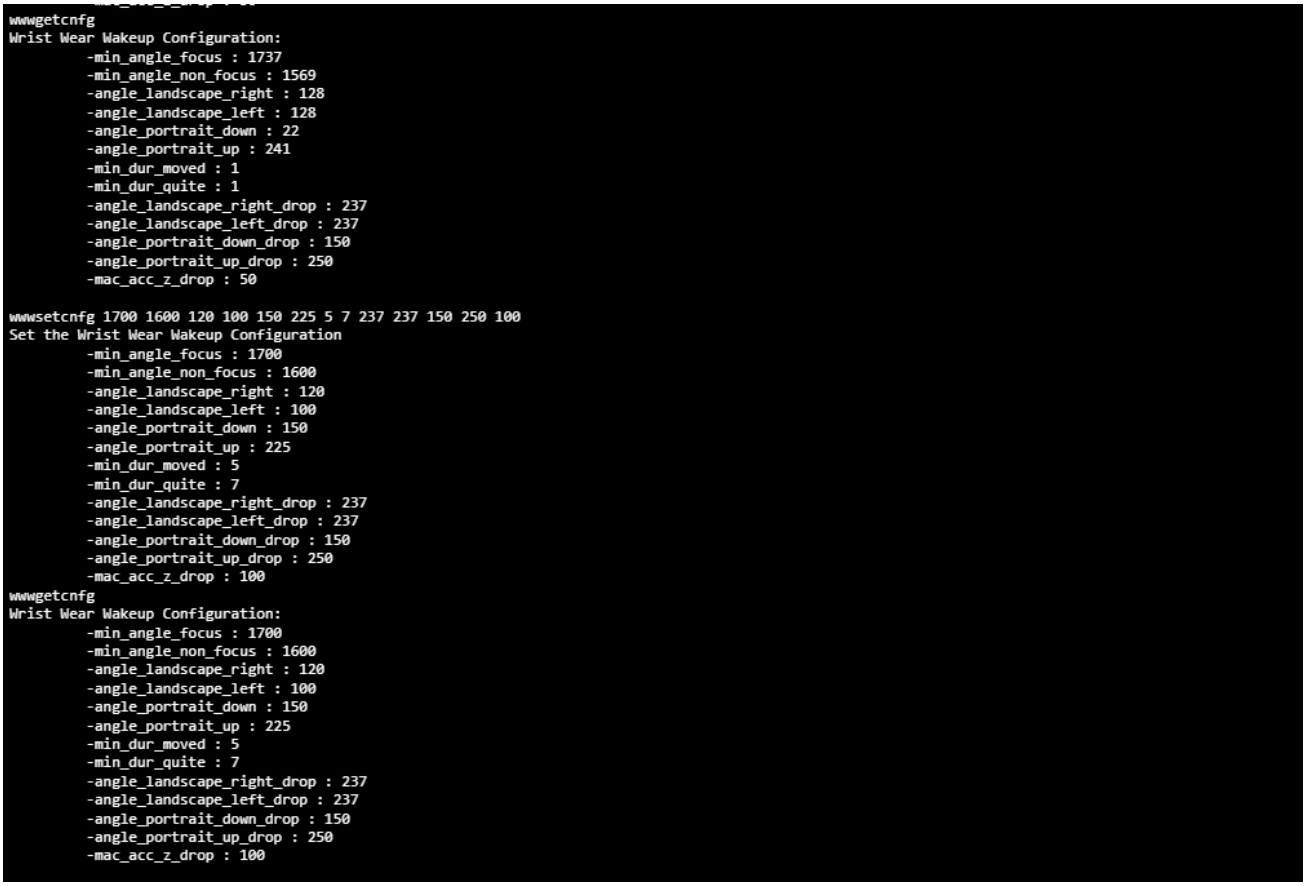

Figure 55: Wrist Wear Wakeup Control Parameter Commands Output

3.13.5 AnyMotion/NoMotion

The following commands allow the user to configure the AnyMotion/NoMotion sensor.

Feature Class	*Command*	*Description*
AnyMotion/NoMotion Sensor Control	***amsetcnfg***	*Set the Any Motion Configuration*
	amgetcnfg	*Get the Any Motion Configuration*
	nmsetcnfg	*Set the No Motion Configuration*
	nmgetcnfg	*Get the No Motion Configuration*

Table 47: AnyMotion/NoMotion Control Parameter Commands Overview

The AnyMotion and NoMotion sensors share same set of parameters. These include:

Configuration	*Description*
duration	*Defines the number of consecutive data points for which the threshold condition must be respected, for interrupt assertion.* *It is expressed in 50Hz samples (20ms). Range is 0-163sec*
axis selection	*Minimum threshold for flick peak on z-axis.* *Scaling: 0.4883, Range: 0x1F4 to 0x5DC*
threshold	*Slope threshold value for any-motion/no-motion detection.* *Range is 0 to 1g.*

Table 48: AnyMotion/NoMotion Control Parameters

AnyMotion/NoMotion Parameter commands usage -	
Command	***Usage***
amsetcnfg	*amsetcnfg <duration> <axis> <threshold>* *eg: amsetcnfg 5 7 170*

amgetcnfg	*amgetcnfg*
nmsetcnfg	*nmsetcnfg <duration> <axis> <threshold>* *eg: nmsetcnfg 1 7 144*
nmgetcnfg	*nmgetcnfg*

Table 49: AnyMotion/NoMotion Control Parameters Usage

```
amgetcnfg
Any Motion Configuration:
        -duration : 16
        -axis_sel : 4
        -threshold : 44

amsetcnfg 5 7 170
Set the Any Motion Configuration
        -duration : 5
        -axis_sel : 7
        -threshold : 170

amgetcnfg
Any Motion Configuration:
        -duration : 5
        -axis_sel : 7
        -threshold : 170

nmgetcnfg
No Motion Configuration:
        -duration : 1
        -axis_sel : 2
        -threshold : 120

nmsetcnfg 1 7 144
Set the No Motion Configuration
        -duration : 1
        -axis_sel : 7
        -threshold : 144
```

Figure 56: AnyMotion/NoMotion Control Parameter Commands Output

3.13.6 Barometer Pressure Type 1/Type 2

The following commands allow the user to configure the Barometer Pressure type 1/type 2 sensor.

Feature Class	*Command*	*Description*
Barometer Pressure Sensor Control	***baro1setcnfg***	*Set the Barometer Pressure Type 1 Configuration*
	baro1getcnfg	*Get the Barometer Pressure Type 1 Configuration*
	baro2setcnfg	*Set the Barometer Pressure Type 2 Configuration*
	baro2getcnfg	*Get the Barometer Pressure Type 2 Configuration*

Table 50: Barometer Pressure Control Parameter Commands Overview

Barometer Pressure Parameter commands usage -	
Command	***Usage***
baro1setcnfg	*baro1setcnfg <osr_p> <osr_t> <iir_filter>* *eg: baro1setcnfg 1 0 1*
baro1getcnfg	*baro1getcnfg*
baro2setcnfg	*baro2setcnfg <osr_p> <osr_t> <iir_filter_p> <iir_filter_t> <dsp_config>* *eg: baro2setcnfg 1 0 1 1 1*
baro2getcnfg	*baro2getcnfg*

Table 51: Barometer Pressure Control Parameter Commands Usage

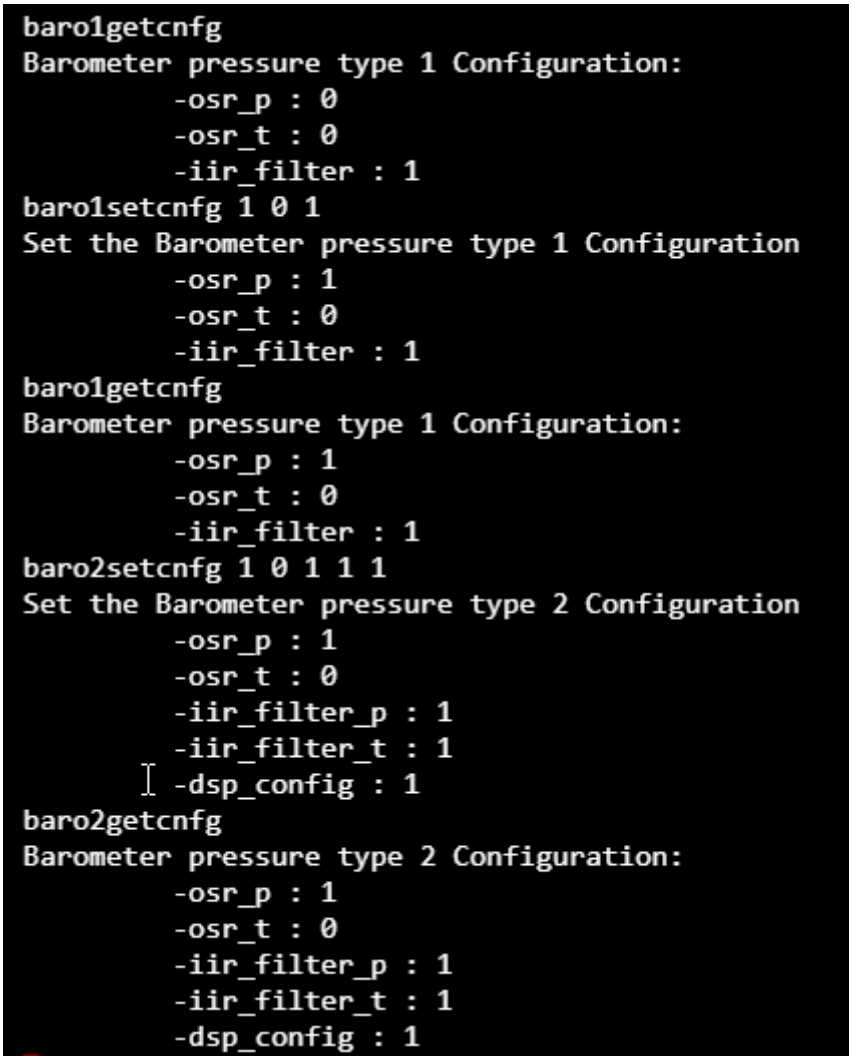

Figure 57: Barometer Pressure Control Parameter Commands Output

3.13.7 Step Counter

The following commands allow the user to configure the Step Counter sensor.

Feature Class	*Command*	*Description*
Step Counter Sensor Control	***scsetcnfg***	*Set the Step Counter Configuration*
	scgetcnfg	*Get the Step Counter Configuration*

Table 52: Step Counter Control Parameter Commands Overview

Step Counter Parameter commands usage -	
Command	***Usage***
scsetcnfg	*scsetcnfg <emdu> <ecu> <emdd> <ecd> <sbs> <mvd> <msd> <fcb2> <fcb1> <fcb0> <fca2> <fca1> <fce> <pdmw> <pdmr> <sdm> <sdw> <hse> <adf> <adt> <sci> <sdpe> <sdt> <empp> <mt>* *eg: scsetcnfg 200 31700 315 31541 5 31551 27856 1220 2430 1220 -6000 17932 1 40 25 150 160 1 12 15600 256 1 3 1 15*
scgetcnfg	*scgetcnfg*

Table 53: Step Counter Control Parameter Commands Usage

To set step counter configuration with values, usage:

```
scsetcnfg 200 31700 315 31541 5 31551 27856 1220 2430 1220 -6000 17932 1 40 25 150 160 1 12 15600 256 1 3 1 15
Set the Step Counter Configuration
        -env_min_dist_up: 200
        -env_coef_up: 31700
        -env_min_dist_down: 315
        -env_coef_down: 31541
        -step_buffer_size: 5
        -mean_val_decay: 31551
        -mean_step_dur: 27856
        -filter_coeff_b2: 1220
        -filter_coeff_b1: 2430
        -filter_coeff_b0: 1220
        -filter_coeff_a2: -6000
        -filter_coeff_a1: 17932
        -filter_cascade_enabled: 1
        -peak_duration_min_walking: 40
        -peak_duration_min_running: 25
        -step_duration_max: 150
        -step_duration_window: 160
        -half_step_enabled: 1
        -activity_detection_factor: 12
        -activity_detection_thres: 15600
        -step_counter_increment: 256
        -step_duration_pp_enabled: 1
        -step_dur_thres: 3
        -en_mcr_pp: 1
        -mcr_thres: 15
```

To get step counter configuration, usage:

```
scgetcnfg
Step Counter Configuration:
        -env_min_dist_up: 200
        -env_coef_up: 31700
        -env_min_dist_down: 315
        -env_coef_down: 31541
        -step_buffer_size: 5
        -mean_val_decay: 31551
        -mean_step_dur: 27856
        -filter_coeff_b2: 1220
        -filter_coeff_b1: 2430
        -filter_coeff_b0: 1220
        -filter_coeff_a2: -6000
        -filter_coeff_a1: 17932
        -filter_cascade_enabled: 1
        -peak_duration_min_walking: 40
        -peak_duration_min_running: 25
        -step_duration_max: 150
        -step_duration_window: 160
        -half_step_enabled: 1
        -activity_detection_factor: 12
        -activity_detection_thres: 15600
        -step_counter_increment: 256
        -step_duration_pp_enabled: 1
        -step_dur_thres: 3
        -en_mcr_pp: 1
        -mcr_thres: 15
```

Figure 58: Step Counter Control Parameter Commands Output

3.13.8 Physical Range

The following commands allow the user to set the physical range of each sensor.

Feature Class	*Command*	*Description*
Physical Range Configuration	***phyrangeconf***	*Set the range of physical sensor*

Table 54: Physical Range Configuration Commands Overview

Physical Range Configuration commands usage -	
Command	***Usage***
phyrangeconf	*phyrangeconf <sensor_id> <range_value>* *eg: phyrangeconf 1 0x10*

Table 55: Physical Range Configuration Commands Usage

To set the range for physical sensor id 1 and range value is 0x10, usage:

```
phyrangeconf 1 0x10
Setting the range of sensor successfully
```

Figure 59: Physical Range Configuration Commands Output

3.13.9 Fast Offset Compensation

The following commands allow the user to enable the fast offset compensation for the sensor.

Feature Class	***Command***	***Description***
Fast Offset Compensation	***foc***	*Enable the fast offset compensation for the sensor*

Table 56: Fast Offset Compensation Commands Overview

Fast Offset Compensation commands usage -	
Command	***Usage***
foc	*foc <sensor id ('1' for accelerometer, '3' for gyroscope)>* *eg: foc 1*

Table 57: Fast Offset Compensation Commands Usage

To enable the fast offset compensation for sensor id 1, usage:

```
foc 1
Keep the sensor stable for accel foc
FOC Success
```

Figure 60: Fast Offset Compensation Commands Output

3.13.10 Static calibration

The following commands allow the user to enable the static calibration for the sensor.

Feature Class	***Command***	***Description***
Static calibration	***staticcalib***	*Enable the fast offset compensation for the sensor*
	getstaticcalib	*Get accel/gyro foc and gyro CRT, store in file*
	setstaticcalib	*Take value from file, set accel/gyro foc and gyro CRT*

Table 58: Static calibration overview

Static calibration commands usage -	
Command	***Usage***
staticcalib	*staticcalib* *eg: staticcalib*
getstaticcalib	*getstaticcalib <file>* *eg: getstaticcalib calib.txt*
setstaticcalib	*setstaticcalib <file>* *eg: setstaticcalib calib.txt*

Table 59: Static calibration command usage

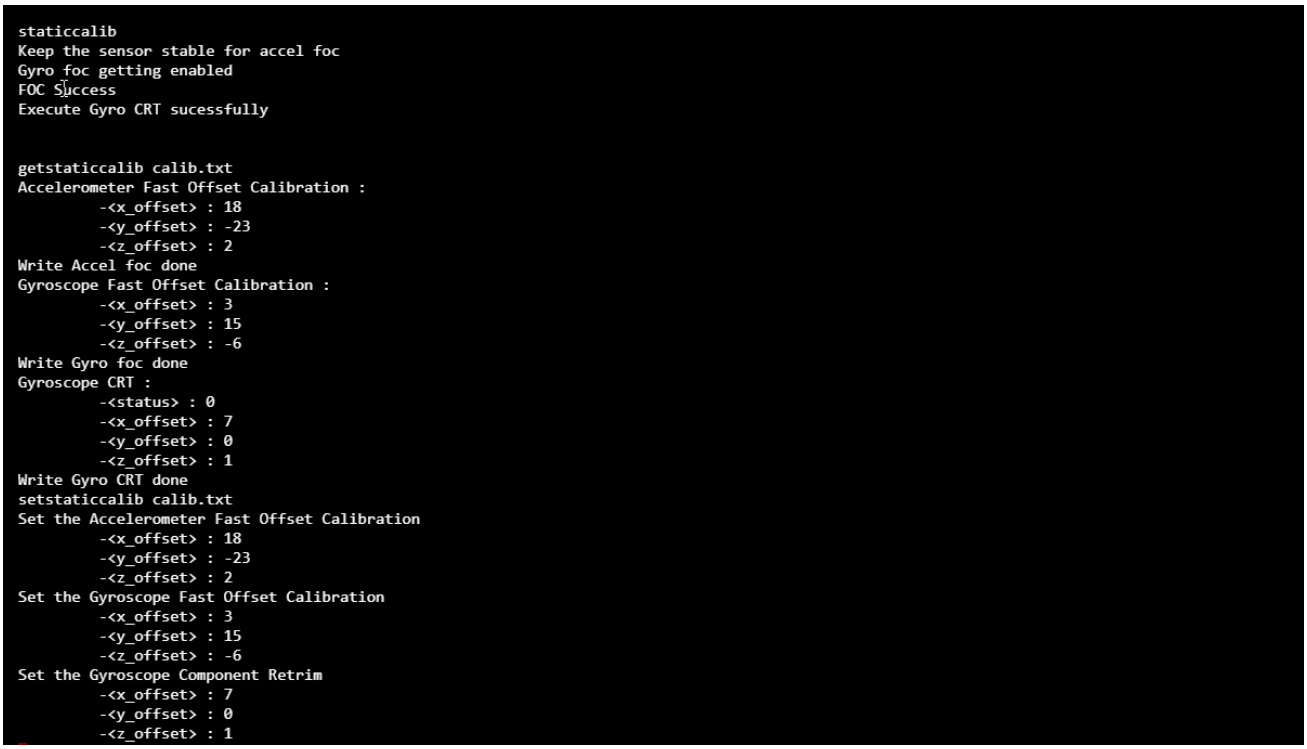

Figure 61: static calibration output

3.14 Generic Features Parameters Commands

3.14.1 Wrist Gesture Detector

The following commands allow the user to configure the Wrist Gesture Detector sensor.

Feature Class	*Command*	*Description*
Wrist Gesture Detector Sensor Control	***wgdsetcnfg***	*Set the Wrist Gesture Detector Configuration*
	wgdgetcnfg	*Get the Wrist Gesture Detector Configuration*
	wgdresetcnfg	*Reset the Wrist Gesture Detector Configuration*

Table 60: Wrist Gesture Detector Control Parameter Commands Overview

Wrist Gesture Detector Parameter commands usage -	
Command	***Usage***
wgdsetcnfg	*wgdsetcnfg <mfpy_th> <mfpz_th> <gx_pos> <gx_neg> <gy_neg> <gz_neg> <fpdc> <lmfc> <mdjp> <dp>,* *eg: wgdsetcnfg 0x400 0x200 0x600 0xf000 0xa000 0x900 0x4000 0x6000 0xb 1*
wgdgetcnfg	*wgdgetcnfg*
wgdresetcnfg	*wgdresetcnfg*

Table 61: Wrist Gesture Detector Control Parameter Commands Usage

```
wgdgetcnfg
Wrist Gesture Detector Configuration:
         -min_flick_peak_y_thres : 0x0640
         -min_flick_peak_z_thres : 0x02bc
         -gravity_bounds_x_pos : 0x0784
         -gravity_bounds_x_neg : 0xfc00
         -gravity_bounds_y_neg : 0xfc3f
         -gravity_bounds_z_neg : 0xf912
         -flick_peak_decay_coeff : 0x7852
         -lp_mean_filter_coeff : 0x7d71
         -max_duration_jiggle_peaks : 0x0010
         -device_pos : 0x00
wgdsetcnfg 0x400 0x200 0x784 0xfc00 0xfc3f 0xf912 0x7852 0x7d71 0x10 0x1
Set the Wrist Gesture Detector Configuration
         -min_flick_peak_y_thres : 0x0400
         -min_flick_peak_z_thres : 0x0200
         -gravity_bounds_x_pos : 0x0784
         -gravity_bounds_x_neg : 0xfc00
         -gravity_bounds_y_neg : 0xfc3f
         -gravity_bounds_z_neg : 0xf912
         -flick_peak_decay_coeff : 0x7852
         -lp_mean_filter_coeff : 0x7d71
         -max_duration_jiggle_peaks : 0x0010
         -device_pos : 0x01
wgdgetcnfg
Wrist Gesture Detector Configuration:
         -min_flick_peak_y_thres : 0x0400
         -min_flick_peak_z_thres : 0x0200
         -gravity_bounds_x_pos : 0x0784
         -gravity_bounds_x_neg : 0xfc00
         -gravity_bounds_y_neg : 0xfc3f
         -gravity_bounds_z_neg : 0xf912
         -flick_peak_decay_coeff : 0x7852
         -lp_mean_filter_coeff : 0x7d71
         -max_duration_jiggle_peaks : 0x0010
         -device_pos : 0x01
wgdresetcnfg
Reset the Wrist Gesture Detector Configuration
```

Figure 62: Wrist Gesture Detector Parameter Commands Output

3.15 Activity Recognition Parameters Commands

The following commands allow the host to set or get Activity Recognition parameters.

Feature Class	Command	Description
Activity Recognition Parameters	***sethearactvcnfg***	*Set the Hearable Activity Configuration*
	gethearactvcnfg	*Get the Hearable Activity Configuration*
	setwearactvcnfg	*Set the Wearable Activity Configuration*
	getwearactvcnfg	*Get the Wearable Activity Configuration*

Table 62: Activity Recognition Parameters Commands Overview

Activity Recognition Parameters commands usage -	
Command	***Usage***
sethearactvcnfg	*sethearactvcnfg <ss> <ppe> <mingt> <maxgt> <obs> <msmc>* *eg. sethearactvcnfg 1 1 1761 2662 1 1*
gethearactvcnfg	*gethearactvcnfg*
setwearactvcnfg	*setwearactvcnfg <ppe> <mingt> <maxgt> <obs> <msmc>* *eg. setwearactvcnfg 1 1761 2662 1 1*
getwearactvcnfg	*getwearactvcnfg*

Table 63: Activity Recognition Parameters Commands Usage

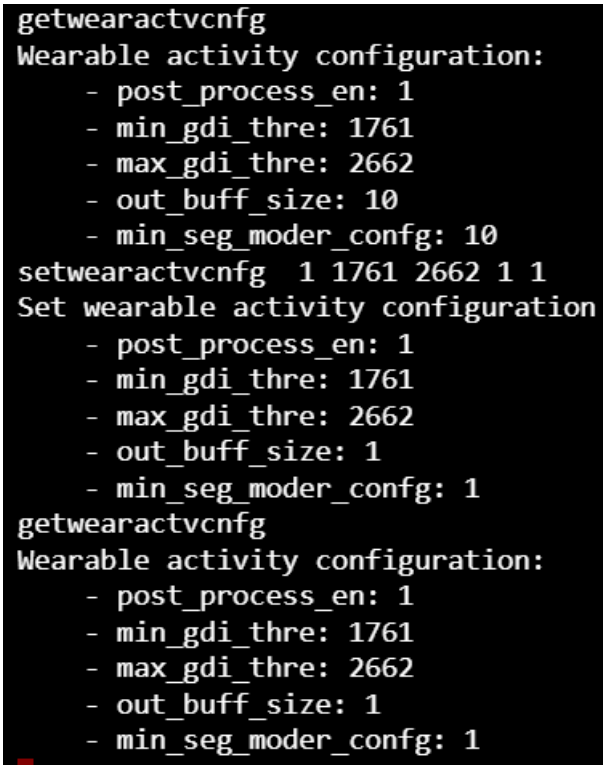

Figure 63: Wearable Activity Recognition Parameters Commands Output

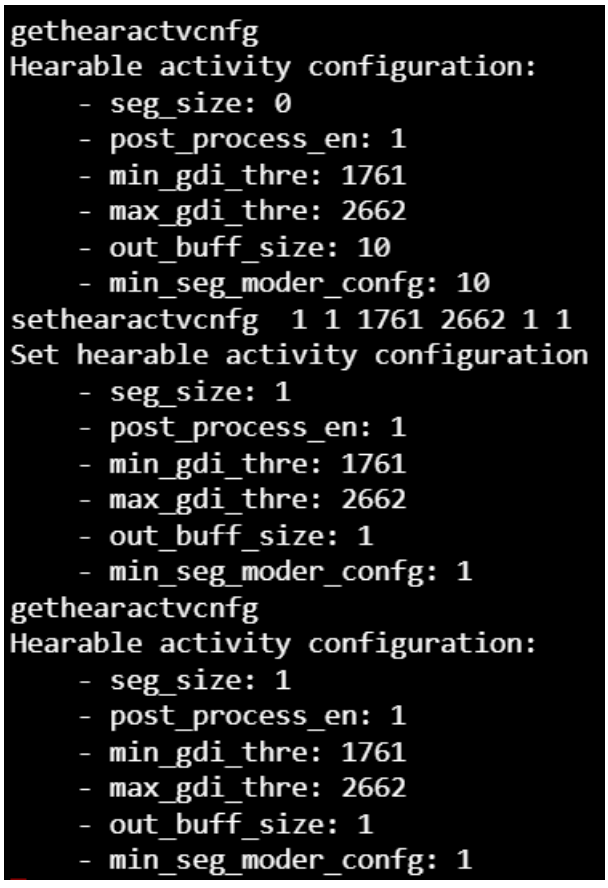

Figure 64: Hearable Activity Recognition Parameters Commands Output

3.16 Data Injection Commands

The following commands allow the host to set or inject data.

Feature Class	Command	Description
Data Injection	***dmode***	*Set the Data Injection mode*
	dinject	*Compute virtual sensor output from raw IMU data*

Table 64: Data Injection Commands Overview

Data Injection commands usage -	
Command	***Usage***
dmode	*dmode <mode>* *eg. dmode s*
dinject	*dinject <input_file.txt>* *eg. dinject field_log.txt*

Table 65: Data Injection Commands Usage

```
Sensor ID: 129, sample rate: 4.000000 Hz, latency: 0 ms


Opened Log File baro_chg.bin.csv_Injection.txt

File Size : 29624
[D][META EVENT]; T: 0.347015625; Flush complete for sensor id 129
[D][META EVENT]; T: 0.347875000; Power mode changed for sensor id 129
[D][META EVENT]; T: 0.347875000; Sample rate changed for sensor id 129
[D]SID: 129; T: 0.250000000; 90297.000000
[D]SID: 129; T: 0.500000000; 90299.000000
[D]SID: 129; T: 0.750000000; 90300.000000
[D]SID: 129; T: 1.000000000; 90299.000000
[D]SID: 129; T: 1.250000000; 90299.000000
[D]SID: 129; T: 1.500000000; 90299.000000
[D]SID: 129; T: 1.750000000; 90299.000000
[D]SID: 129; T: 2.000000000; 90301.000000
[D]SID: 129; T: 2.250000000; 90300.000000
[D]SID: 129; T: 2.500000000; 90300.000000
[D]SID: 129; T: 2.750000000; 90299.000000
[D]SID: 129; T: 3.000000000; 90299.000000
[D]SID: 129; T: 3.250000000; 90295.000000
[D]SID: 129; T: 3.500000000; 90298.000000
[D]SID: 129; T: 3.750000000; 90296.000000
[D]SID: 129; T: 4.000000000; 90297.000000
[D]SID: 129; T: 4.250000000; 90298.000000
[D]SID: 129; T: 4.500000000; 90298.000000
[D]SID: 129; T: 4.750000000; 90299.000000
[D]SID: 129; T: 5.000000000; 90298.000000
[D]SID: 129; T: 5.250000000; 90297.000000
[D]SID: 129; T: 5.500000000; 90299.000000
[D]SID: 129; T: 5.750000000; 90299.000000
[D]SID: 129; T: 6.000000000; 90301.000000
[D]SID: 129; T: 6.250000000; 90300.000000
[D]SID: 129; T: 6.500000000; 90301.000000
[D]SID: 129; T: 6.750000000; 90299.000000
[D]SID: 129; T: 7.000000000; 90300.000000
[D]SID: 129; T: 7.250000000; 90301.000000
[D]SID: 129; T: 7.500000000; 90299.000000
[D]SID: 129; T: 7.750000000; 90301.000000
[D]SID: 129; T: 8.000000000; 90301.000000
[D]SID: 129; T: 8.250000000; 90301.000000
[D]SID: 129; T: 8.500000000; 90300.000000
[D]SID: 129; T: 8.750000000; 90298.000000
[D]SID: 129; T: 9.000000000; 90298.000000
[D]SID: 129; T: 9.250000000; 90296.000000
[D]SID: 129; T: 9.500000000; 90297.000000
[D]SID: 129; T: 9.750000000; 90299.000000
[D]SID: 129; T: 10.000000000; 90298.000000
[D]SID: 129; T: 10.250000000; 90299.000000
[D]SID: 129; T: 10.500000000; 90298.000000
[D]SID: 129; T: 10.750000000; 90298.000000
[D]SID: 129; T: 11.000000000; 90298.000000
[D]SID: 129; T: 11.250000000; 90297.000000
[D]SID: 129; T: 11.500000000; 90298.000000
[D]SID: 129; T: 11.750000000; 90299.000000
[D]SID: 129; T: 12.000000000; 90298.000000
[D]SID: 129; T: 12.250000000; 90298.000000
[D]SID: 129; T: 12.500000000; 90299.000000
[D]SID: 129; T: 12.750000000; 90299.000000
```

Figure 65: Data Injection Commands Output

3.17 Diagnostics Commands

The following commands allow the host to get diagnostics information when the fatal error happens in BHy.

Feature Class	Command	Description
Diagnostics	***-m OR postm***	*Get Post Mortem Data and log to a file*

Table 66: Diagnostics Command Overview

Diagnostics commands usage -	
Command	***Usage***
postm	*postm <pm_log_filename.bin>* *eg. postm pm_log_filename.bin*

Table 67: Diagnostics Command Usage

```
Error Reg Value : 44
POST MORTEM STATUS :
valid           :       0x00000001
flags           :       0x00000007

CONTEXT :
reg_1           :       0x00000000
reg_2           :       0x00000000
reg_3           :       0x00a15238
reg_4           :       0x00000024
reg_5           :       0x00a04b90
reg_6           :       0x00f0000c
reg_7           :       0x00000000
reg_8           :       0x00200000
reg_9           :       0x0000003f
reg_10          :       0x10101010
reg_11          :       0x00002300
reg_12          :       0x00a15638
reg_13          :       0x00000000
reg_14          :       0x00a042b4
reg_15          :       0x00a04e04
reg_16          :       0x00000001
reg_17          :       0x00a111c0
reg_18          :       0x00000001
reg_19          :       0x00000008
reg_20          :       0x00a042c0
reg_21          :       0x21212121
reg_22          :       0x22222222
reg_23          :       0x23232323
reg_24          :       0x24242424
reg_25          :       0x25252525
reg_26          :       0x00a05c1c
gp    [reg_27]  :       0x00a1117c
fp    [reg_28]  :       0x00a11158
sp    [reg_29]  :       0x8000481e
ilink [reg_30]  :       0x30303030
reg_31          :       0x0011ad74
blink [reg_32]  :       0x001029f2

SYSTEM STATUS :
eret            :       0x0010e562
erbta           :       0x0010e4bc
erstatus        :       0x8000461e
ecr             :       0x00020000
efa             :       0x0010e562
icause          :       0x00000000
mpu_ecr         :       0x00000000

DIAGNOSTIC :
diagnostic      :       0x00000002
debug state     :       0x000000b1
debug val       :       0x00000000
error val       :       0x00000000
interrupt       :       0x00000000
err report      :       0x00000044

STACK INFO :
stack start     :       0x00a05c20
stack size      :       0x00001000
```

Figure 66: Diagnostics Command Output

3.18 Utility Commands

The utility commands are application specific commands and allow the users to control and manage the various application specific configurations to have an enhanced user interaction.

Feature Class	Utility	Command	Description
Utility	**Debug**	**-v OR verb**	Set the verbose level.
	Status	**echo**	Enable/Disable echo
		heart	Enable/disable Heartbeat Message
	File	**mklog**	Create a log file
		rm	Remove a log file
		ls	List the files in the external Flash
		wrfile	Write to a log file
		rdfile	Read from a log file
		slabel	Set a string label in the log file
	UI	**cls**	Clear Screen

Table 68: Utility Commands Overview

Note: *Except the 'verb' command, all other utility commands are exclusive to MCU mode.*

3.18.1 Debug Utility

The verbose command '-v' or 'verb' sets the verbose level. The verbose level refers to the level of response that the user expects from the application regarding its state of execution. The verbose level is classified in the following table.

verbose	*scope*
0	*Give only error notifications.*
1	*Give notifications regarding errors as well as warnings.*
2	*Give notifications regarding the complete state of the system in terms of errors, warnings, and information about the current state of execution.*

Table 69: Verbose Levels

Debug Utility -	
Command	**Usage**
verb	*verb <verbose_level>, eg: verb 1*

Table 70: Debug Utility Command Usage

```
version
HW info:: Board: 5, HW ID: 11, Shuttle ID: 179, SW ID: 10
SW Version: 0.6.0
Build date: May 20 2025


verb 2
[I]Executing verb 2
Setting verbose to 2


version
[I]Executing version
HW info:: Board: 5, HW ID: 11, Shuttle ID: 179, SW ID: 10
SW Version: 0.6.0
Build date: May 20 2025
```

Figure 67: Debug Utility Commands Output

By default, the verbose level is set to 0 to limit the number of notifications to need-to-know. The verbose level can be configured accordingly based on debugging and application requirements.

3.18.2 Status Utility

The echo command echoes back the input given to the application. The echo command can be used to enable/disable the echo feature.
The 'heart' command is used to indicate the system notifications to the user by blinking the LED every time a notification is sent. The heart command can be used to enable/disable the heartbeat feature.

Status Utility -	
Command	**Usage**
echo	*echo <state>, eg: echo on*
heart	*heart <state>, eg: heart off*

Table 71: Status Utility Commands Usage

```
echo off
Setting echo to off


Setting echo to on


heart on
Setting Heartbeat message to on


[H]34807
[H]34857
[H]37307
[H]37357
[H]39807
[H]39857

[H]42307
[H]42357
heart off
Setting Heartbeat message to off
```

Figure 68: Status Utility Commands Output

3.18.3 File Utility

The File utility commands allow the user to handle the various File and Log operations such as create/delete file, read/write file, add annotations etc. The file utility commands are different from the logging commands described in **Log Generation Commands**, in the sense that the log commands are more oriented towards logging the sensor data, whereas the File utility commands are more oriented towards facilitating the user with file management of the external Flash on the application board.

The following table describes those commands:

File Utility -	
Action	**Commands**
Generate the log file	*mklog <file_name.file_extension>, eg: mklog abc.txt*
List the files in the external Flash of the application board	*ls*
Write to the log file	*wrfile <file_name.file_extension> <length_in_bytes>, eg: wrfile abc.txt 150* ***Note 1**: In an event, a file is not present with the passed filename, a new file with same filename is generated.* ***Note 2**: There is a input timeout duration of 10s. After executing the 'wrfile' command, if no input is provided within the 10s, the application will assert the 'wrfile' callback and return 'Write Timed Out' error.* ***Note 3**: If the same command executed second time. The new input data will be appended to the previous content of the file.* ***Note 4**: While uploading the data from a PC using any application, ensure that <size_of_file> is passed as the input to length argument, and not the <size_of_file_on_disk>*
Read the log file	*rdfile <file_name.file_extension, eg: rdfile abc.txt*
Delete the File	*rm <file_name.file_extension>, eg: rm abc.txt*

Annotate the log file	*slabel <label string>, eg: slabel Activity_N* ***Note**: slabel command can be used in association with Log generation commands, when acquiring the data in logging mode to annotate the events.*

Table 72: File Utility Commands Usage

```
attlog log.udf
File log.udf was created


logse 1:100
Sensor ID: 1, sample rate: 100.000000 Hz, latency: 0 ms


[D][META EVENT]; T: 17.266484375; Power mode changed for sensor id 1
[D][META EVENT]; T: 17.266484375; Sample rate changed for sensor id 1
[D][META EVENT]; T: 17.265906250; Flush complete for sensor id 1

slabel start
slabel end
detlog log.udf
File log.udf was detached for logging
```

Figure 69: slabel output

Note: Switching application board to MTP mode to get **log.udf** file, then using **udf2csv** tool to convert it to csv file for convenient observation.

3.18.3.1 File Handling

```
ls
                            README.TXT | 731 B


mklog ab.txt
File ab.txt was created


ls
                            README.TXT | 731 B
                                ab.txt | 0 B


wrfile abc.txt 10

Waiting for 10 bytes of data
File Transferred : 100.00%
Write Completed


rdfile abc.txt
Read Initiated
abcdef0123
Read Completed

rm ab.txt
File ab.txt was removed


ls
                            README.TXT | 731 B
                               abc.txt | 10 B
```

Figure 70: File Utility Commands Output

3.18.3.2 Annotation

Annotation is very useful from a data collection perspective. It is used to annotate the various events covered during the scope of data collection.

File Annotation	
Action	**Usage**

Attach a log file for the logging	*attlog <file_name.file_extension>, eg: attlog abc.bin*
Enable the sensor acquisition in logging mode	*logse <id>:<odr>, eg: logse 4:100*
Annotate the log file	*slabel <label string>, eg: slabel Activity_N*
Disable the sensor acquisition	*logse <id>:0, eg: logse 4:0*
Detach log file from logging	*detlog <file_name.file_extension>, eg: detlog abc.bin*

Table 73: File Annotation using 'slabel'

3.18.4 UI Utility

The 'cls' is a UI command and used to clear the screen of the UI.

Debug Utility	
Command	**Usage**
cls	*cls*

Table 74: 'cls' Command Usage

4 BHy2CLI Limitations

4.1 HW Limitations

- There is a limitation on the file name length when transferring files to APP30 External Flash in MTP mode. It allows a maximum of 150 characters for file names.

4.2 Platform Limitations

- Copying of the log files using Windows Explorer can lead to data corruption.
- For Head Orientation sensors, the ODR change is not reflected for subsequent sensor activation with different ODRs.
- Due to filesystem constraints, it does not support the simultaneous writing of multiple files to the external memory of application boards.
- In MCU mode, a file will be appended instead of removed and/or created as a new one for all commands (except 'rm' and 'getbsxparam') with handling file.
- HEX streaming only supports ODR with a frequency not bigger than 400Hz.
- Due to algorithm, sometimes, BSX magnetometer parameters are not restored properly when the sensors are enabled with logse/actse command

5 Legal Disclaimer

5.1 Engineering Samples

Engineering Samples are marked with an asterisk (*) Or (e). Samples may vary from the valid technical specifications of the product series contained in the user guide. They are therefore not intended or fit for resale to third parties or for use in end products. Their sole purpose is internal client testing. The testing of an engineering sample may in no way replace the testing of a product series. Bosch Sensortec assumes no liability for the use of engineering samples. The Purchaser shall indemnify Bosch Sensortec from all claims arising from the use of engineering samples.

5.2 Product Use

Bosch Sensortec products are developed for the consumer goods industry. They are not designed or approved for use in military applications, life-support appliances, safety-critical automotive applications and devices or systems where malfunctions of these products can reasonably be expected to result in personal injury. They may only be used within the parameters of this user guide.

The resale and/or use of products are at the Purchaser’s own risk and the Purchaser’s own responsibility.

The Purchaser shall indemnify Bosch Sensortec from all third party claims arising from any product use not covered by the parameters of this user guide or not approved by Bosch Sensortec and reimburse Bosch Sensortec for all costs in connection with such claims.

The Purchaser accepts the responsibility to monitor the market for the purchased products, particularly with regard to product safety, and inform Bosch Sensortec without delay of any security relevant incidents.

5.3 Application examples and hints

With respect to any examples or hints given herein, any typical values stated herein and/or any information regarding the application of the device, Bosch Sensortec hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights or copyrights of any third party. The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics. They are provided for illustrative purposes only and no evaluation regarding infringement of intellectual property rights or copyrights or regarding functionality, performance or error has been made.

6 References

Reference 1: BHy2xx/BHI3xx Evaluation Setup Guide
Reference 2: BHI360 and BHI385 Datasheet

Document number: BST-MSS-SD005-30
Revision_3.0_082026

7 Document history and modifications

Rev. No	Chapter	Description of modification/changes	Date
1.0		First Draft	Nov 2023
1.1		Addressed the Review Comments	Jan 2024
2.0		Added new commands for Parameters, Info Added new chapter for limitations	Nov 2024
2.1		Updated compatibility Corrected some typos in commands	Apr 2025
2.2		Updated compatibility Changed name from BHyCLI to BHy2CLI Corrected some typos in commands Removed commands related to Flash Updated limitation related to BSX magnetometer	May 2025
2.3		Update image, descriptions for some commands Remove descriptions related to **strbuf** command	June 2025
2.4		Added cyclic kilo commands	August 2025
2.5		Added generic kilo commands	October 2025
2.6		Added steps to clone, compile BHy2CLI tool Removed generic kilo commands	November 2025
2.7		Updated coines, Sensor APIs, support Hearable device	December 2025
2.8		Updated content	March 2026
3.0		Updated content, support BHI360 v2.3.1	August 2026

Bosch Sensortec GmbH
Gerhard-Kindler-Straße 9
72770 Reutlingen / Germany

www.bosch-sensortec.com

Document number: BST-MSS-SD005-30
Revision_3.0_082026